

SYSTEM S3 — SOFTWARE DESIGN DOCUMENT

Inventory & purchasing, the restaurant and F&B point of sale, and hotel rooms, reservations & folios — the system that runs the guest-facing and back-of-house operations of the hotel

CONSOLIDATES FORMER MODULES

M5 Inventory & Purchasing Service

M6 Restaurant & F&B Service

M7 Hotel & Room Management Service

DOCUMENT INFORMATION

Document type	Software Design Document — Developer Edition
Intended audience	The developer (or team) building System 3 — Hospitality Operations. Read S0 first.
Version	1.0
Date	2026-06-15
Companion documents	S0 Shared Platform Reference · S1 Identity & Access Platform · S2 Workforce & Payroll · S4 Finance & BI

Table of Contents

1. Purpose, Scope & Responsibilities	4
1.1 Why S3 exists	4
1.2 Out of scope	4
1.3 Relationship to the original M5 / M6 / M7 services	5
2. Internal Architecture	7
2.1 Functional areas	7
2.2 The ID3 / ID4 / ID5 simplifications	8
2.3 Folder structure	10
3. Data Model	12
3.1 Inventory & Purchasing module	12
3.2 Restaurant & F&B module	18
3.3 Hotel & Room Management module	23
3.4 Platform plumbing tables	28
4. API Endpoint Catalog	29
4.1 Inventory & Purchasing module	29
4.2 Restaurant & F&B module	30
4.3 Hotel & Room Management module	31
5. Core Business Logic	33
5.1 Order-to-folio sequence overview	34
5.2 FIFO / FEFO stock rotation (ID4)	35
5.3 PO approval thresholds	36
5.4 Service charge and VAT formula	37
5.5 Customer types and post-payment billing routing	37
5.6 Folio charges and check-in / check-out logic	38
5.7 Partial folio payment management	38
5.8 Employee F&B consumption push to payroll (ID12)	39
6. Role-Based Access Control	40
6.1 Permission catalog	40

6.2 System roles — S3 access summary	41
6.3 CheckPermission middleware contract	42
6.4 Permission seeding	42
7. Inventory & Hospitality Lifecycle	43
7.1 Section A — Purchase order approval chain (<code>purchase_orders.status</code>)	43
7.2 Section B — Stock batch lifecycle (<code>stock_batches.status</code>)	44
7.3 Section C — Restaurant order and bill lifecycle	45
7.4 Section D — Reservation, room & folio lifecycle	46
8. Event-Driven Integration	47
8.1 Events emitted by S3	47
8.2 Events consumed by S3	47
8.3 Synchronous calls	48
8.4 Outbox mechanics	48
8.5 Idempotency mechanics (D5)	48
9. Procurement, Supplier Payments, Mandatory Invoice Fields & Cashier Reconciliation	50
9.1 Purchase order and goods receipt workflow	50
9.2 Supplier payment modes and GL postings	50
9.3 Mandatory folio invoice fields	51
9.4 Cashier shift reconciliation	52
10. Security, Secrets & Operational Conventions	53
10.1 Platform conventions (D1–D14) as applied to S3	53
10.2 Environment configuration (<code>.env</code>)	54
11. Non-Functional Requirements & Testing	55
12. Integration Contract Summary	57
12.1 What S3 provides	57
12.2 What S3 depends on	57
12.3 Build-order implication	58

NOTE

This document is the complete build specification for **System 3 — Hospitality Operations**, one Laravel 11 application backed by one MySQL database (`wh_s3_db`). S3 consolidates three former microservices — **M5 Inventory & Purchasing**, **M6 Restaurant & F&B**, and **M7 Hotel & Room Management** — into a single modular monolith with three internal modules: **Inventory & Purchasing**, **Restaurant & F&B**, and **Hotel & Room Management**.

The three biggest simplifications from this merger are **ID4** (ingredient dispatch on order finalization becomes an in-process call to `FifoFefoService::dispatch()`), **ID5** (posting a restaurant charge to a guest's room folio becomes an in-process call to `FolioService::postChargeForRoom()`), and **ID3** (S3 nets hotel-guest F&B revenue against the room folio internally and sends a single clean journal entry to S4, eliminating the old “wash account”). Together these remove three of the busiest cross-service calls in the original nine-module design.

Before reading further, make sure you have read **S0 — Cross-System Integration & Platform Conventions**, which defines the gateway, the JWT flow, the cross-system event bus, the permission-naming convention (ID9), and the platform conventions (D1–D14) referenced throughout this document by ID.

1. Purpose, Scope & Responsibilities

1.1 Why S3 exists

S3 is the **operational heart of the hotel** — it owns everything that happens between a supplier delivering goods and a guest checking out. Concretely, S3 owns:

- **Inventory & Purchasing** — the item catalogue, stock batches with FIFO/FEFO rotation, stock movements, the purchase-order approval chain, suppliers, goods receipts, and supplier payments.
- **Restaurant & F&B** — the menu, recipes (which tie menu items to inventory items), dining tables, restaurant orders and order items, bills with mandatory service-charge/VAT calculation, bill payments across seven customer types, and the staff F&B consumption ledger.
- **Hotel & Room Management** — room types and rooms, guest profiles, reservations, room folios, folio charges (room rent, F&B, laundry, minibar, event, other), folio payments, and cashier shifts.

All of the above lives in **27 tables** inside `wh_s3_db` (10 Inventory & Purchasing, 9 Restaurant & F&B, 8 Hotel & Room Management), plus the two platform plumbing tables (`event_outbox`, `idempotency_keys`). S3 is the system that every receptionist, cashier, waiter, inventory officer, and restaurant manager interacts with every working hour, and it is the single largest source of journal entries and operational events feeding S4.

1.2 Out of scope

CONVENTION

S3 does not do any of the following — these responsibilities belong to other systems, and S3 only interacts with them through the contracts defined in S0.

Responsibility	Owned by	How S3 interacts with it
Authentication, JWT issuance/verification, RBAC enforcement	S1	Every inbound request carries a JWT verified by the gateway (S0 §3–4); S3 only checks <code>permissions[]</code> claims via <code>CheckPermission</code> middleware.
Employee master data, display names, departments	S2	S3 stores only an <code>employee_id</code> / <code>cashier_id</code> / <code>created_by</code>

Responsibility	Owned by	How S3 interacts with it
		reference (an integer pointing at S2's <code>wh_s2_db.employees.id</code>) — never a copy of the employee's name, role, or department. Display names are resolved via S1's <code>name</code> JWT claim (ID2) or S4's cached <code>GET /api/v1/employees</code> read.
Staff payroll deductions for F&B consumption	S2	S3 accumulates <code>employee_consumption</code> internally and pushes the period total to S2 via <code>POST /employees/{id}/deductions</code> (ID12, S0 §5.3).
Chart of accounts, ledger, financial statements	S4	S3 posts goods-receipt/AP, F&B-sales, room-revenue/folio-settlement, and event-AR journal entries via <code>POST /api/v1/journal-entries</code> (S0 §5.3) but stores no account balances itself.
BI dashboards, KPI aggregation, scheduled reports	S4	S4 reads S3's rooms, reservations, orders, items, and purchase-order data via cached <code>GET</code> endpoints (S0 §5.3–5.4, 60–120s TTL).
Any user interface	—	Out of scope for every system in this architecture; S3 exposes a JSON API only.

1.3 Relationship to the original M5 / M6 / M7 services

If you previously read the M5, M6, or M7 microservice specs, the following table summarises what changed when they were merged into S3.

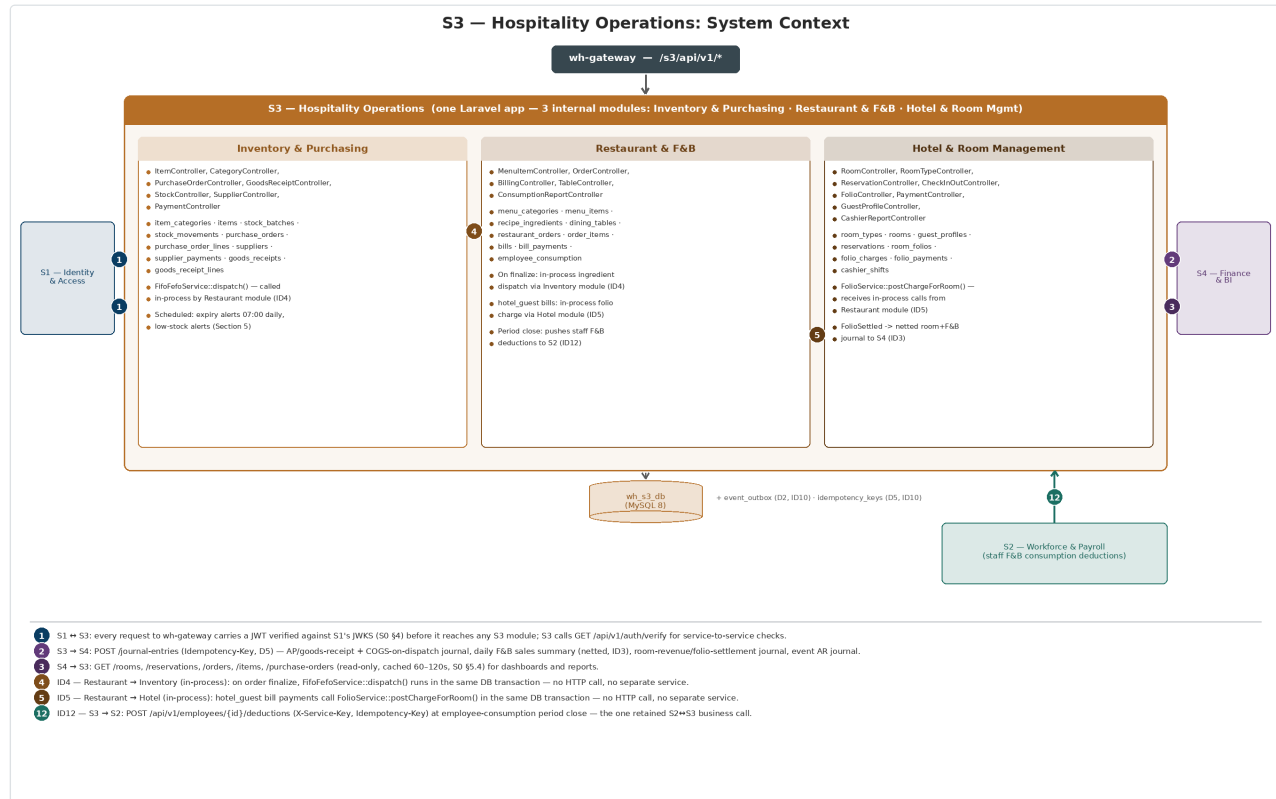
Area	Former (M5/M6/M7)	Now (S3)
Ingredient dispatch on order finalize	M6 called <code>M5 POST /items/dispatch</code> over HTTP for every recipe ingredient	ID4 — <code>OrderController::finalize()</code> calls the Inventory module's <code>FifoFefoService::dispatch()</code> directly, in the same DB transaction as the bill computation; no HTTP, no retry policy, no partial-failure window
Posting an F&B charge to a hotel guest's folio	M6 called <code>M7 GET /active-folio</code> then <code>POST /fnb-charge</code> over HTTP (Decision B4 in the original design)	ID5 — <code>BillingController::pay()</code> 's <code>routeByCustomerType()</code> calls the Hotel module's <code>FolioService::postChargeForRoom()</code> directly, in the same DB transaction
GL postings for hotel-guest F&B	M6 and M7 each posted independently to M8, requiring a “wash account” (<code>4002.1 F&B Revenue – Room-Billed</code>) in M9's	ID3 — S3 nets hotel-guest F&B revenue against the room folio internally and sends one journal entry to S4 per settled folio; the daily

Area	Former (M5/M6/M7)	Now (S3)
	reporting layer to avoid double-counting	F&B sales summary journal excludes <code>hotel_guest</code> orders entirely (Section 8.1, S0 §8.1)
Cross-system messaging	Each service had its own ad-hoc webhook conventions	Unified <code>event_outbox</code> table + <code>PublishOutboxEvents</code> job (D2, ID10)
Idempotent writes	Not modelled	<code>idempotency_keys</code> table (D5, ID10) protects <code>PUT /purchase-orders/{id}/approve</code> , <code>POST /suppliers/{id}/payments</code> , <code>POST /bills/{id}/payments</code> , <code>POST /folios/{id}/payments</code> , and the outbound <code>POST /employees/{id}/deductions</code> to S2
Permission names	<code>M5.*</code> , <code>M6.*</code> , <code>M7.*</code>	Renamed to <code>S3.inventory.*</code> , <code>S3.restaurant.*</code> (also <code>S3.fnb.*</code> where the original used “F&B”), <code>S3.hotel.*</code> per ID9’s <code>S{n}.{domain}.{resource}.{action}</code> convention
Cashier reconciliation across F&B and rooms	Not modelled — M6 and M7 cashiers reconciled separately	New <code>cashier_shifts</code> table (Section 3.4); <code>bill_payments.shift_id</code> (Restaurant) and <code>folio_payments.shift_id</code> (Hotel) both reference it, so one cashier’s combined F&B + room takings reconcile against one shift record
Staff F&B consumption → payroll	M6 → M3 <code>POST /deductions</code>	Retained as the one S3 ↔ S2 business call (ID12), now <code>POST /api/v1/employees/{id}/deductions</code> , Idempotency-Key protected, Service-Key authenticated

...

2. Internal Architecture

S3 is organised into three Laravel modules under a single application and database. The diagram below shows the module boundaries, their primary controllers and tables, the two in-process cross-module calls (ID4, ID5), and how S3 talks to S1, S2, and S4.



S3 system context diagram showing a single Laravel application containing three module sub-boxes — Inventory & Purchasing (ItemController, PurchaseOrderController, GoodsReceiptController, StockController, SupplierController, and related tables), Restaurant & F&B (OrderController, BillingController, MenuItemController, and related tables), and Hotel & Room Management (ReservationController, CheckInOutController, FolioController, CashierReportController, and related tables). Dashed badge 4 marks the in-process Restaurant-to-Inventory arrow (ID4, FifoFifoService::dispatch()); dashed badge 5 marks the in-process Restaurant-to-Hotel arrow (ID5, FolioService::postChargeForRoom()). A wh_s3_db cylinder sits below the three modules. Badge 1 connects S1 for JWT auth; badge 2 connects S3 to S4 for journal entries; badge 3 connects S4 to S3 for cached reads; badge 12 connects S3 to S2 for staff meal deductions (ID12). A six-item legend explains each connection.

2.1 Functional areas

Inventory & Purchasing module

Concern	Controllers	Tables
Item catalogue & categories	ItemController, CategoryController	item_categories, items
Stock batches & movements	StockController	stock_batches, stock_movements
Purchase orders & approval chain	PurchaseOrderController	purchase_orders, purchase_order_lines

Concern	Controllers	Tables
Goods receipts	<code>GoodsReceiptController</code>	<code>goods_receipts</code> , <code>goods_receipt_lines</code> , <code>stock_batches</code>
Suppliers & payments	<code>SupplierController</code> , <code>PaymentController</code>	<code>suppliers</code> , <code>supplier_payments</code>
Stock alerts (expiry, low-stock)	<code>StockController</code> (scheduled commands)	<code>items</code> , <code>stock_batches</code>

Restaurant & F&B module

Concern	Controllers	Tables
Menu catalogue & recipes	<code>MenuItemController</code>	<code>menu_categories</code> , <code>menu_items</code> , <code>recipe_ingredients</code>
Dining tables	<code>TableController</code>	<code>dining_tables</code>
Orders & order items	<code>OrderController</code>	<code>restaurant_orders</code> , <code>order_items</code>
Billing & payments	<code>BillingController</code>	<code>bills</code> , <code>bill_payments</code>
Staff F&B consumption	<code>ConsumptionReportController</code>	<code>employee_consumption</code>

Hotel & Room Management module

Concern	Controllers	Tables
Room types & rooms	<code>RoomTypeController</code> , <code>RoomController</code>	<code>room_types</code> , <code>rooms</code>
Guest profiles	<code>GuestProfileController</code>	<code>guest_profiles</code>
Reservations & check-in/out	<code>ReservationController</code> , <code>CheckInOutController</code>	<code>reservations</code> , <code>rooms</code> , <code>room_folios</code>
Folios, charges & payments	<code>FolioController</code> , <code>PaymentController</code>	<code>room_folios</code> , <code>folio_charges</code> , <code>folio_payments</code>
Cashier shifts & reconciliation	<code>CashierReportController</code>	<code>cashier_shifts</code> , <code>folio_payments</code> , <code>bill_payments</code> (Restaurant module)

2.2 The ID3 / ID4 / ID5 simplifications

In the original three-microservice design, a single hotel-guest restaurant order touched all three of M5, M6, and M7 over HTTP, and M6's and M7's revenue postings to M8 had to be reconciled in M9 via a “wash account.” In S3, all three of those interactions are now in-process method calls inside one Laravel application and one database transaction:

- **ID4 — ingredient dispatch.** `OrderController::finalize()` iterates every `order_items` row whose `menu_items.has_recipe = true`, and for each `recipe_ingredients` row calls the Inventory module's `FifoFefoService::dispatch($item, $quantity)` directly (Section 5.2). The dispatch decrements `stock_batches.quantity_remaining` and `items.current_stock`, writes a `stock_movements` row, and accumulates Cost of Food Sold at the dispatched batches' `unit_cost` — all inside the same transaction that creates the `bills` row.

- **ID5 — folio posting.** When `BillingController::pay()` resolves a bill's `customer_type = hotel_guest`, it calls the Hotel module's `FolioService::postChargeForRoom($room_id, $bill)` directly (Section 5.6). This resolves the room's open `room_folios` row, inserts a `folio_charges` row (`service_category = fnb`) with its own SC/VAT snapshot, and increments `room_folios.total_charges` — again, the same transaction.
- **ID3 — netted GL posting.** Because ID4 and ID5 already happen in-process, S3 can compute the *correct, non-overlapping* revenue figures before ever calling S4. `FolioService::recordPayment()` posts one netted journal entry per settled folio (room rent + any F&B charged to that folio), and the separate daily F&B sales-summary journal **excludes `hotel_guest` orders entirely** — their revenue is already in the folio-settlement journal. S4 therefore receives two clean, non-overlapping journals and needs no “wash account” netting convention (S0 §8.1).

There is no network call, no serialization, and no possibility of one module succeeding while another is unreachable for any of these three interactions. Section 5 walks through the full order-to-folio sequence step by step.

2.3 Folder structure

```

app/
├── Http/
│   ├── Controllers/
│   │   ├── Inventory/
│   │   │   ├── ItemController.php
│   │   │   ├── CategoryController.php
│   │   │   ├── StockController.php
│   │   │   ├── PurchaseOrderController.php
│   │   │   ├── GoodsReceiptController.php
│   │   │   ├── SupplierController.php
│   │   │   └── PaymentController.php
│   │   ├── Restaurant/
│   │   │   ├── MenuItemController.php
│   │   │   ├── TableController.php
│   │   │   ├── OrderController.php
│   │   │   ├── BillingController.php
│   │   │   └── ConsumptionReportController.php
│   │   └── Hotel/
│   │       ├── RoomTypeController.php
│   │       ├── RoomController.php
│   │       ├── GuestProfileController.php
│   │       ├── ReservationController.php
│   │       ├── CheckInOutController.php
│   │       ├── FolioController.php
│   │       ├── PaymentController.php
│   │       └── CashierReportController.php
│   ├── Middleware/
│   │   ├── VerifyJwt.php
│   │   ├── CheckPermission.php
│   │   ├── ServiceKeyAuthenticate.php
│   │   └── EnsureIdempotent.php
│   └── Models/
│       ├── Inventory/
│       │   ├── ItemCategory.php
│       │   ├── Item.php
│       │   ├── StockBatch.php
│       │   ├── StockMovement.php
│       │   ├── PurchaseOrder.php
│       │   ├── PurchaseOrderLine.php
│       │   ├── GoodsReceipt.php
│       │   ├── GoodsReceiptLine.php
│       │   ├── Supplier.php
│       │   └── SupplierPayment.php
│       ├── Restaurant/
│       │   ├── MenuCategory.php
│       │   ├── MenuItem.php
│       │   ├── RecipeIngredient.php
│       │   ├── DiningTable.php
│       │   ├── RestaurantOrder.php
│       │   ├── OrderItem.php
│       │   ├── Bill.php
│       │   ├── BillPayment.php
│       │   └── EmployeeConsumption.php
│       └── Hotel/
│           ├── RoomType.php
│           ├── Room.php
│           ├── GuestProfile.php
│           ├── Reservation.php
│           └── RoomFolio.php

```

```

├── FolioCharge.php
├── FolioPayment.php
├── CashierShift.php
├── EventOutbox.php
├── IdempotencyKey.php
├── Services/
│   ├── Inventory/
│   │   └── FifoFefoService.php
│   ├── Restaurant/
│   │   ├── BillingService.php
│   │   └── EmployeeConsumptionService.php
│   ├── Hotel/
│   │   └── FolioService.php
│   └── FinanceClientService.php
├── Events/
│   ├── GoodsReceived.php
│   ├── PurchaseOrderApproved.php
│   ├── OrderFinalized.php
│   ├── EmployeeConsumptionPeriodClosed.php
│   ├── GuestCheckedIn.php
│   ├── GuestCheckedOut.php
│   └── FolioSettled.php
├── Listeners/
│   └── PublishLifecycleEventToOutbox.php
├── Jobs/
│   ├── PublishOutboxEvents.php
│   ├── PurgeIdempotencyKeys.php
│   ├── RaiseExpiryAlerts.php
│   └── RaiseLowStockAlerts.php
├── app/Console/Commands/
│   ├── RaiseExpiryAlerts.php          (daily 07:00)
│   └── RaiseLowStockAlerts.php
├── database/migrations/    (27 business tables + event_outbox + idempotency_keys – see Section 3)
├── routes/api.php
├── config/
└── .env

```

NOTE

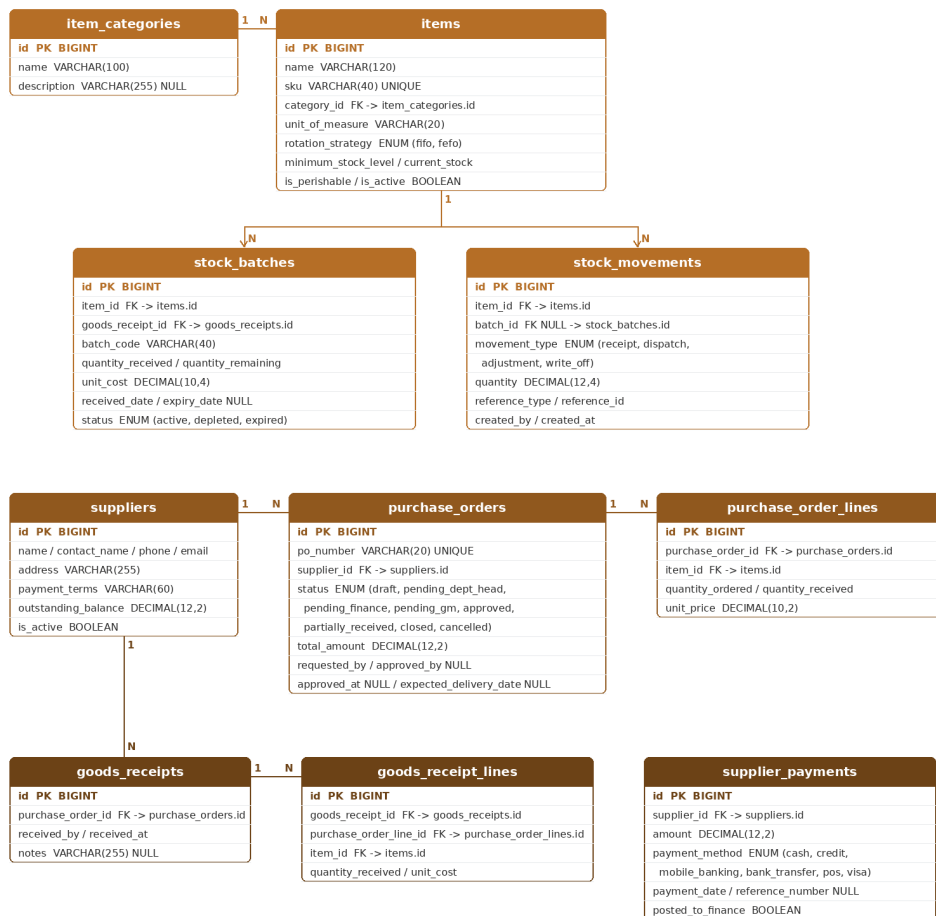
`FinanceClientService` is the single outbound HTTP client used by all three S3 modules — it wraps `POST /api/v1/journal-entries` to S4 (Section 5, Section 8.3) with the Service-Key header and Idempotency-Key required by D5. `FifoFefoService` (Inventory) and `FolioService` (Hotel) are called **in-process** by the Restaurant module's `OrderController` and `BillingController` (ID4, ID5) — they are plain service classes, not HTTP clients, and run inside the same database transaction as the caller.

3. Data Model

S3 owns 27 business tables in `wh_s3_db`, grouped into Inventory & Purchasing (10 tables), Restaurant & F&B (9 tables), Hotel & Room Management (8 tables), and platform plumbing (2 tables). The three ER diagrams below show each module's tables; `items` appears in both the Inventory diagram (as the canonical table) and the Restaurant diagram (as a dashed reference card), since `recipe_ingredients.item_id` is a real foreign key that crosses the module boundary inside the same database.

3.1 Inventory & Purchasing module

S3 — Inventory & Purchasing Module: Entity-Relationship Diagram



Legend:

PK = primary key · FK = foreign key · 1 / N = relationship cardinality. All ten tables live in `wh_s3_db`, Inventory & Purchasing module.

Not shown (to reduce clutter): `purchase_orders.id -> goods_receipts.purchase_order_id` (1:N, partial deliveries create multiple receipts); `purchase_order_lines.id -> goods_receipt_lines.purchase_order_line_id` (1:N); `goods_receipts.id -> stock_batches.goods_receipt_id` (1:1 per receipt line, `GoodsReceiptController::store()` creates one `stock_batches` row per `goods_receipt_line`); `stock_batches.id -> stock_movements.batch_id` (1:N, nullable for adjustments).

Cross-module FK (real FK, same database): Restaurant module's `recipe_ingredients.item_id -> items.id` (see `s3_er_restaurant.png`).

Entity-relationship diagram of the ten Inventory and Purchasing tables in `wh_s3_db`. The top row shows `item_categories` on the left connected by a 1-to-N relationship to `items` in the centre. From `items`, fan connections drop to `stock_batches` and `stock_movements` in the second row. The third row shows `suppliers` connected by 1-to-N to `purchase_orders`, which connects by 1-to-N to `purchase_order_lines`. The fourth row shows `goods_receipts` connected by 1-to-N to `goods_receipt_lines`, with `supplier_payments` alongside also connected to `suppliers` by a 1-to-N relationship. All ten tables are rendered in amber tones and live in `wh_s3_db`, Inventory and Purchasing module. A legend notes: the `purchase_orders` to `goods_receipts` 1-to-N relationship (partial deliveries can create multiple receipts) is not drawn to reduce clutter, nor is the `purchase_order_lines` to `goods_receipt_lines` link or the `goods_receipts` to `stock_batches` 1-to-1-per-line relationship. A cross-module FK note states that `recipe_ingredients.item_id` references `items.id` as a real FK in the same database.

item_categories

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(100)	e.g. “Dry Goods”, “Dairy & Chilled”, “Beverages”, “Cleaning Supplies”
<code>description</code>	VARCHAR(255) NULL	

items

The items table is the catalogue of every stockable input. `recipe_ingredients.item_id` in the Restaurant module holds a real FK into this table.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(120)	
<code>sku</code>	VARCHAR(40), UNIQUE	Stock-keeping unit code
<code>category_id</code>	BIGINT, FK → <code>item_categories.id</code>	
<code>unit_of_measure</code>	VARCHAR(20)	e.g. <code>kg</code> , <code>litre</code> , <code>piece</code>
<code>rotation_strategy</code>	ENUM(<code>fifo</code> , <code>fefo</code>)	Drives dispatch order in <code>FifoFefoService</code> — see Section 5.2
<code>minimum_stock_level</code>	DECIMAL(10,2)	Alert threshold for the low-stock scheduled job
<code>current_stock</code>	DECIMAL(12,2)	Running total; decremented on dispatch, incremented on goods receipt
<code>is_perishable</code>	BOOLEAN	<code>true</code> = has expiry dates on stock batches
<code>is_active</code>	BOOLEAN	Default <code>true</code> ; inactive items are excluded from ordering and dispatch

stock_batches

One row per goods-receipt line. `GoodsReceiptController::store()` creates one `stock_batches` row for every `goods_receipt_lines` row when a goods receipt is recorded.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>item_id</code>	BIGINT, FK → <code>items.id</code>	
<code>goods_receipt_id</code>	BIGINT, FK → <code>goods_receipts.id</code>	
<code>batch_code</code>	VARCHAR(40)	Supplier batch reference or auto-generated code
<code>quantity_received</code>	DECIMAL(12,2)	Immutable after creation — the original received quantity

Column	Type	Notes
<code>quantity_remaining</code>	DECIMAL(12,2)	Decrement on each <code>FifoFefoService::dispatch()</code> call
<code>unit_cost</code>	DECIMAL(10,4)	Cost per unit at receipt time — used for COGS valuation (S0 \$8)
<code>received_date</code>	DATE	Used for FIFO ordering (<code>received_date ASC</code>)
<code>expiry_date</code>	DATE NULL	Used for FEFO ordering (<code>expiry_date ASC</code>); <code>NULL</code> for non-perishables
<code>status</code>	ENUM(<code>active</code> , <code>depleted</code> , <code>expired</code>)	See Section 7 lifecycle

stock_movements

One row for every change to inventory — receipt, dispatch, adjustment, or write-off.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>item_id</code>	BIGINT, FK → <code>items.id</code>	
<code>batch_id</code>	BIGINT, FK NULL → <code>stock_batches.id</code>	NULL for write-offs and manual adjustments not tied to a specific batch
<code>movement_type</code>	ENUM(<code>receipt</code> , <code>dispatch</code> , <code>adjustment</code> , <code>write_off</code>)	
<code>quantity</code>	DECIMAL(12,4)	Positive = stock in, negative = stock out
<code>reference_type</code>	VARCHAR(50)	e.g. <code>goods_receipt</code> , <code>restaurant_order</code> , <code>adjustment</code>
<code>reference_id</code>	BIGINT	ID of the source record
<code>created_by</code>	BIGINT	S2 <code>employees.id</code> reference (not a real FK — cross-database)
<code>created_at</code>	TIMESTAMP	

suppliers

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(150)	
<code>contact_name</code>	VARCHAR(100) NULL	
<code>phone</code>	VARCHAR(20) NULL	
<code>email</code>	VARCHAR(120) NULL	
<code>address</code>	VARCHAR(255) NULL	

Column	Type	Notes
<code>payment_terms</code>	VARCHAR(60) NULL	e.g. "Net 30"
<code>outstanding_balance</code>	DECIMAL(12,2)	Running AP balance; incremented on goods receipt, decremented on supplier payment
<code>is_active</code>	BOOLEAN	Inactive suppliers cannot receive new POs

purchase_orders

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>po_number</code>	VARCHAR(20), UNIQUE	Auto-generated, e.g. <code>PO-0001</code>
<code>supplier_id</code>	BIGINT, FK → <code>suppliers.id</code>	
<code>status</code>	ENUM(<code>draft</code> , <code>pending_dept_head</code> , <code>pending_finance</code> , <code>pending_gm</code> , <code>approved</code> , <code>partially_received</code> , <code>closed</code> , <code>cancelled</code>)	See Section 7.1 lifecycle
<code>total_amount</code>	DECIMAL(12,2)	Sum of <code>purchase_order_lines.quantity_ordered × unit_price</code> ; drives the approval-tier selection
<code>requested_by</code>	BIGINT	S2 <code>employees.id</code> reference
<code>approved_by</code>	BIGINT NULL	Final approver's S2 <code>employees.id</code>
<code>approved_at</code>	TIMESTAMP NULL	
<code>expected_delivery_date</code>	DATE NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

purchase_order_lines

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>purchase_order_id</code>	BIGINT, FK → <code>purchase_orders.id</code>	
<code>item_id</code>	BIGINT, FK → <code>items.id</code>	
<code>quantity_ordered</code>	DECIMAL(12,2)	
<code>quantity_received</code>	DECIMAL(12,2)	Cumulative total across all <code>goods_receipt_lines</code> for this PO line; when it equals <code>quantity_ordered</code> the line is fully received
<code>unit_price</code>	DECIMAL(10,2)	Agreed price per unit with the supplier

goods_receipts

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>purchase_order_id</code>	BIGINT, FK → <code>purchase_orders.id</code>	
<code>received_by</code>	BIGINT	S2 <code>employees.id</code> reference
<code>received_at</code>	TIMESTAMP	
<code>notes</code>	VARCHAR(255) NULL	Delivery notes or discrepancy remarks

NOTE

`GoodsReceiptController::store()` runs in a single transaction: it creates the `goods_receipts` row, creates one `goods_receipt_lines` row per item, creates one `stock_batches` row per `goods_receipt_lines` row, increments `items.current_stock`, increments `purchase_order_lines.quantity_received`, updates `suppliers.outstanding_balance`, sets `purchase_orders.status` to `partially_received` or `closed` depending on whether all lines are now fully received, writes a `stock_movements` row (`movement_type = receipt`) per item, and writes a `wh.events.s3.goods.received` row to `event_outbox`.

goods_receipt_lines

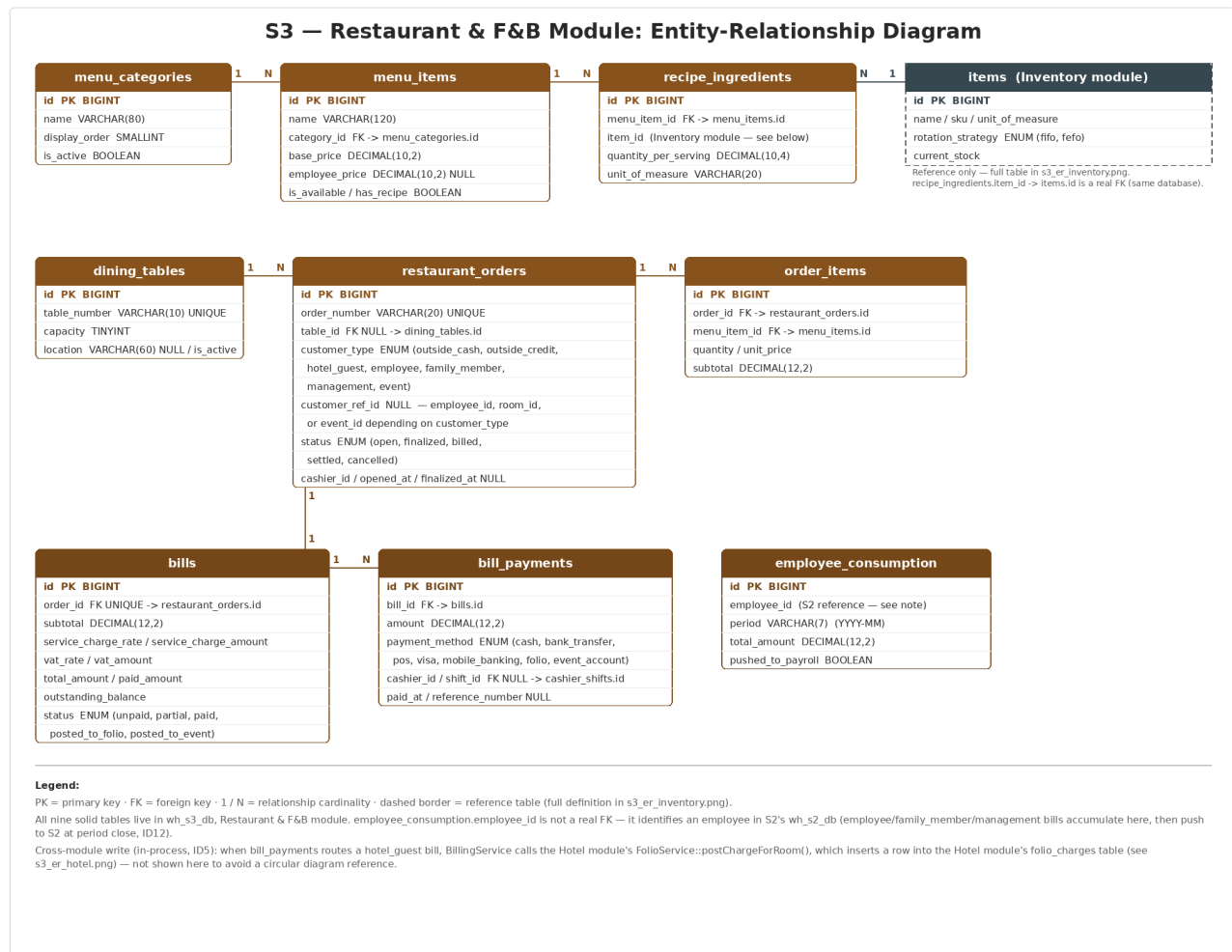
Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>goods_receipt_id</code>	BIGINT, FK → <code>goods_receipts.id</code>	
<code>purchase_order_line_id</code>	BIGINT, FK → <code>purchase_order_lines.id</code>	
<code>item_id</code>	BIGINT, FK → <code>items.id</code>	Denormalised for convenience; must match the PO line's <code>item_id</code>
<code>quantity_received</code>	DECIMAL(12,2)	Actual quantity received in this delivery
<code>unit_cost</code>	DECIMAL(10,4)	Actual unit cost on this delivery (may differ from the PO's <code>unit_price</code>)

supplier_payments

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>supplier_id</code>	BIGINT, FK → <code>suppliers.id</code>	
<code>amount</code>	DECIMAL(12,2)	
<code>payment_method</code>	ENUM(<code>cash</code> , <code>credit</code> , <code>mobile_banking</code> , <code>bank_transfer</code> , <code>pos</code> , <code>visa</code>)	See Section 9.2 for GL posting per method
<code>payment_date</code>	DATE	
<code>reference_number</code>	VARCHAR(80) NULL	

Column	Type	Notes
		Bank reference, mobile transaction ID, POS receipt number, etc.
<code>posted_to_finance</code>	BOOLEAN	<code>true</code> after <code>FinanceClientService</code> successfully posts the AP journal to S4

3.2 Restaurant & F&B module



Entity-relationship diagram of the nine Restaurant and F&B tables in wh_s3_db, plus a dashed reference card for the Inventory module's items table. The top row shows menu_categories on the left connected by 1-to-N to menu_items, which connects by 1-to-N to recipe_ingredients; recipe_ingredients connects by N-to-1 to the dashed items reference card. The second row shows dining_tables on the left connected by 1-to-N to restaurant_orders, which connects by 1-to-N to order_items. The third row shows bills on the left connected by 1-to-1 down from restaurant_orders, then bills connects by 1-to-N to bill_payments. employee_consumption appears alongside bill_payments. A legend notes that all nine solid tables live in wh_s3_db Restaurant and F&B module, that employee_consumption.employee_id is not a real FK (it identifies an employee in S2's wh_s2_db), and that the cross-module write to Hotel's folio_charges via BillingService calling FolioService::postChargeForRoom() (ID5) is not shown to avoid a circular diagram reference.

menu_categories

Column	Type	Notes
id	BIGINT, PK	
name	VARCHAR(80)	e.g. "Breakfast", "Main Course", "Beverages", "Desserts"
display_order	SMALLINT	Controls menu ordering in client-facing displays
is_active	BOOLEAN	Inactive categories are hidden from ordering

menu_items

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(120)	
<code>category_id</code>	BIGINT, FK → <code>menu_categories.id</code>	
<code>base_price</code>	DECIMAL(10,2)	Standard price charged to <code>outside_cash</code> , <code>outside_credit</code> , <code>hotel_guest</code> , and <code>event</code> customers
<code>employee_price</code>	DECIMAL(10,2) NULL	Discounted price charged to <code>employee</code> , <code>family_member</code> , and <code>management</code> customers; if NULL the <code>base_price</code> is used
<code>is_available</code>	BOOLEAN	Toggled by restaurant manager to flag temporary unavailability
<code>has_recipe</code>	BOOLEAN	<code>true</code> = this item has <code>recipe_ingredients</code> rows; <code>FifoFefoService::dispatch()</code> is called for each ingredient on finalize

recipe_ingredients

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>menu_item_id</code>	BIGINT, FK → <code>menu_items.id</code>	
<code>item_id</code>	BIGINT, FK → <code>items.id</code>	Real FK — cross-module, same database. References the Inventory module's <code>items</code> table.
<code>quantity_per_serving</code>	DECIMAL(10,4)	Quantity of the inventory item consumed per serving of this menu item
<code>unit_of_measure</code>	VARCHAR(20)	Must match <code>items.unit_of_measure</code> for the dispatch calculation to be correct

dining_tables

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>table_number</code>	VARCHAR(10), UNIQUE	e.g. "T01", "T12", "BAR-1"
<code>capacity</code>	TINYINT	Maximum covers
<code>location</code>	VARCHAR(60) NULL	e.g. "Main Hall", "Terrace", "Private Room"
<code>is_active</code>	BOOLEAN	Inactive tables cannot receive new orders

restaurant_orders

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>order_number</code>	VARCHAR(20), UNIQUE	Auto-generated
<code>table_id</code>	BIGINT, FK NULL → <code>dining_tables.id</code>	NULL for takeaway, room-service, or event orders
<code>customer_type</code>	ENUM(<code>outside_cash</code> , <code>outside_credit</code> , <code>hotel_guest</code> , <code>employee</code> , <code>family_member</code> , <code>management</code> , <code>event</code>)	Drives price selection and post-payment routing — see Section 5.4
<code>customer_ref_id</code>	BIGINT NULL	The relevant ID for the customer type: <code>rooms.id</code> for <code>hotel_guest</code> ; S2 <code>employees.id</code> for <code>employee</code> / <code>family_member</code> / <code>management</code> ; an event identifier for <code>event</code> ; NULL for <code>outside_*</code> types
<code>status</code>	ENUM(<code>open</code> , <code>finalized</code> , <code>billed</code> , <code>settled</code> , <code>cancelled</code>)	See Section 7.3 lifecycle
<code>cashier_id</code>	BIGINT	S2 <code>employees.id</code> reference — the waiter or cashier who opened the order
<code>opened_at</code>	TIMESTAMP	
<code>finalized_at</code>	TIMESTAMP NULL	Set when <code>OrderController::finalize()</code> runs

order_items

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>order_id</code>	BIGINT, FK → <code>restaurant_orders.id</code>	
<code>menu_item_id</code>	BIGINT, FK → <code>menu_items.id</code>	
<code>quantity</code>	DECIMAL(10,2)	
<code>unit_price</code>	DECIMAL(10,2)	Snapshot of <code>menu_items.base_price</code> or <code>employee_price</code> at time of order, depending on <code>customer_type</code>
<code>subtotal</code>	DECIMAL(12,2)	<code>quantity × unit_price</code>

bills

Each finalized order has exactly one bill (unique FK on `order_id`).

Column	Type	Notes
<code>id</code>	BIGINT, PK	

Column	Type	Notes
<code>order_id</code>	BIGINT, FK UNIQUE → <code>restaurant_orders.id</code>	
<code>subtotal</code>	DECIMAL(12,2)	Sum of <code>order_items.subtotal</code>
<code>service_charge_rate</code>	DECIMAL(5,4)	Rate snapshot at bill-creation time (default 0.10)
<code>service_charge_amount</code>	DECIMAL(12,2)	<code>round(subtotal × service_charge_rate, 2)</code>
<code>vat_rate</code>	DECIMAL(5,4)	Rate snapshot at bill-creation time (default 0.15)
<code>vat_amount</code>	DECIMAL(12,2)	<code>round((subtotal + service_charge_amount) × vat_rate, 2)</code>
<code>total_amount</code>	DECIMAL(12,2)	<code>subtotal + service_charge_amount + vat_amount</code>
<code>paid_amount</code>	DECIMAL(12,2)	Cumulative total across all <code>bill_payments</code> rows
<code>outstanding_balance</code>	DECIMAL(12,2)	<code>total_amount - paid_amount</code> ; recomputed on every payment
<code>status</code>	ENUM(<code>unpaid</code> , <code>partial</code> , <code>paid</code> , <code>posted_to_folio</code> , <code>posted_to_event</code>)	See Section 7.3 lifecycle

bill_payments

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>bill_id</code>	BIGINT, FK → <code>bills.id</code>	
<code>amount</code>	DECIMAL(12,2)	
<code>payment_method</code>	ENUM(<code>cash</code> , <code>bank_transfer</code> , <code>pos</code> , <code>visa</code> , <code>mobile_banking</code> , <code>folio</code> , <code>event_account</code>)	<code>folio</code> and <code>event_account</code> are used when the bill is routed via ID5 or the event-AR mechanism
<code>cashier_id</code>	BIGINT	S2 <code>employees.id</code> reference
<code>shift_id</code>	BIGINT, FK NULL → <code>cashier_shifts.id</code>	Links this payment to the cashier's open shift for reconciliation (Section 9.4)
<code>paid_at</code>	TIMESTAMP	
<code>reference_number</code>	VARCHAR(80) NULL	Bank ref, POS receipt, mobile transaction ID

employee_consumption

One row per employee per billing period. Accumulates the total F&B consumption of staff (`employee`, `family_member`, and `management` customer types) during a calendar month, to be pushed to S2 at period close (ID12).

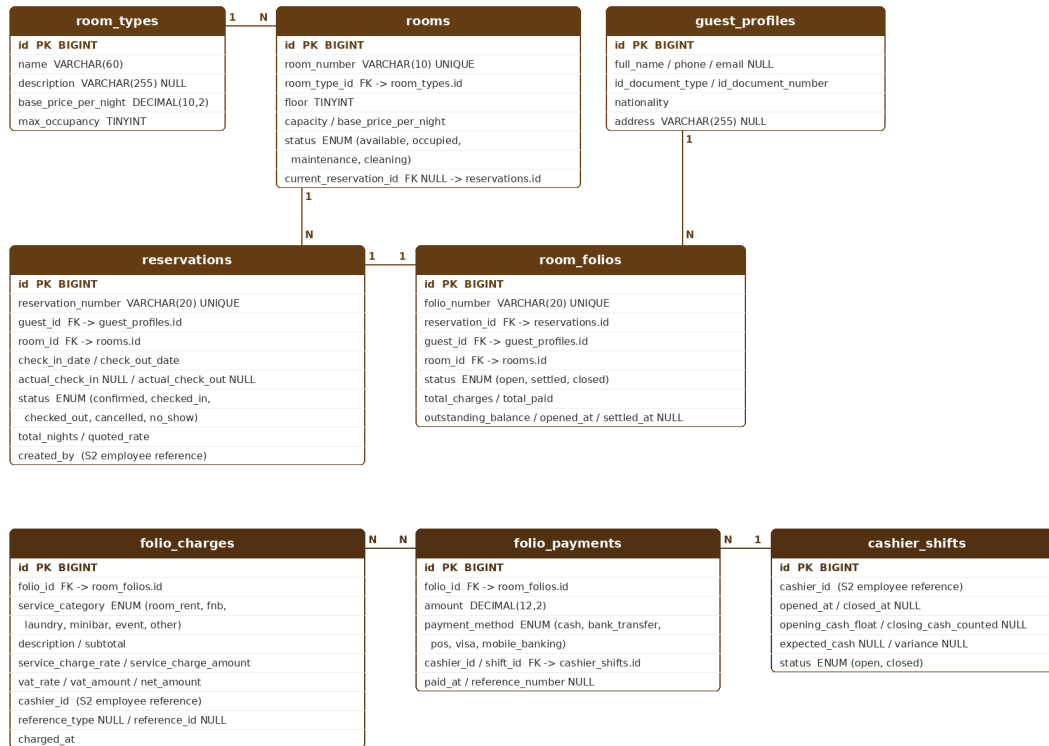
Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT	S2 <code>employees.id</code> reference — not a real FK (cross-database)
<code>period</code>	VARCHAR(7)	<code>YYYY-MM</code> format, e.g. <code>2026-06</code>
<code>total_amount</code>	DECIMAL(12,2)	Running total of billed amounts for the period (pre-SC/VAT, since the deduction is of the food cost, not the full bill)
<code>pushed_to_payroll</code>	BOOLEAN	<code>true</code> after <code>EmployeeConsumptionService::pushToPayroll()</code> successfully calls S2

NOTE

`employee_consumption` has a unique constraint on `(employee_id, period)` — `EmployeeConsumptionService::accumulate()` uses an upsert to increment `total_amount` on every `employee / family_member / management` bill payment. The `pushed_to_payroll` flag ensures the period-close push to S2 is idempotent: if the S2 call fails and is retried, rows already marked `true` are skipped.

3.3 Hotel & Room Management module

S3 — Hotel & Room Management Module: Entity-Relationship Diagram



Legend:

PK = primary key · FK = foreign key · 1 / N = relationship cardinality. All eight tables live in wh_s3_db, Hotel & Room Management module.
 cashier_id / created_by columns are not FKs — they identify an employee in S2's wh_s2_db (resolved via GET /api/v1/employees/{id} for display, S0 §5.3).
 cashier_shifts is new for S3 (Section 3.4): folio_payments.shift_id and bill_payments.shift_id (Restaurant module, s3_er_restaurant.png) both reference it, so a cashier's combined Restaurant + Hotel takings reconcile against one shift record (the consolidated 'cashier' role, S0 §7.2).
 rooms.current_reservation_id is set on check-in and cleared on check-out — a denormalized convenience pointer; the authoritative link is reservations.room_id.
 room_folios -- folio_charges is 1:N (not drawn — the two rows don't align); see folio_charges.folio_id -> room_folios.id.

Entity-relationship diagram of the eight Hotel and Room Management tables in wh_s3_db. The top row shows room_types on the left connected by 1-to-N to rooms, and guest_profiles on the right. The second row shows reservations on the left connected by 1-to-1 to room_folios, with rooms and guest_profiles connecting to reservations and room_folios via vertical connections. The third row shows folio_charges on the left, folio_payments in the centre, and cashier_shifts on the right; folio_payments connects to cashier_shifts by N-to-1. All eight tables live in wh_s3_db, Hotel and Room Management module. A legend notes that cashier_id and created_by columns are not real FKs but identify employees in S2; that cashier_shifts is new for S3 and allows combined Restaurant plus Hotel takings to reconcile against one shift record; that rooms.current_reservation_id is a denormalized convenience pointer; and that the 1-to-N relationship between room_folios and folio_charges is not drawn to avoid layout issues.

room_types

Column	Type	Notes
id	BIGINT, PK	
name	VARCHAR(60)	e.g. "Standard Single", "Deluxe Double", "Suite"
description	VARCHAR(255) NULL	
base_price_per_night	DECIMAL(10,2)	Default nightly rate; can be overridden at reservation time in <code>reservations.quoted_rate</code>

Column	Type	Notes
<code>max_occupancy</code>	TINYINT	Maximum number of guests

rooms

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>room_number</code>	VARCHAR(10), UNIQUE	e.g. “101”, “305A”
<code>room_type_id</code>	BIGINT, FK → <code>room_types.id</code>	
<code>floor</code>	TINYINT	
<code>capacity</code>	TINYINT	Actual capacity (may differ from <code>room_types.max_occupancy</code> for converted rooms)
<code>base_price_per_night</code>	DECIMAL(10,2)	Room-level override of the type's base price; used if set, otherwise falls back to <code>room_types.base_price_per_night</code>
<code>status</code>	ENUM(<code>available</code> , <code>occupied</code> , <code>maintenance</code> , <code>cleaning</code>)	See Section 7.4 lifecycle
<code>current_reservation_id</code>	BIGINT, FK NULL → <code>reservations.id</code>	Denormalized convenience pointer set on check-in and cleared on check-out; the authoritative link is <code>reservations.room_id</code>

guest_profiles

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>full_name</code>	VARCHAR(120)	
<code>phone</code>	VARCHAR(20) NULL	
<code>email</code>	VARCHAR(120) NULL	
<code>id_document_type</code>	VARCHAR(40)	e.g. “Passport”, “National ID”, “Driver's License”
<code>id_document_number</code>	VARCHAR(60)	
<code>nationality</code>	VARCHAR(60)	
<code>address</code>	VARCHAR(255) NULL	

reservations

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>reservation_number</code>	VARCHAR(20), UNIQUE	Auto-generated
<code>guest_id</code>	BIGINT, FK → <code>guest_profiles.id</code>	

Column	Type	Notes
<code>room_id</code>	BIGINT, FK → <code>rooms.id</code>	
<code>check_in_date</code>	DATE	Scheduled check-in date
<code>check_out_date</code>	DATE	Scheduled check-out date
<code>actual_check_in</code>	TIMESTAMP NULL	Set by <code>CheckInOutController::checkIn()</code>
<code>actual_check_out</code>	TIMESTAMP NULL	Set by <code>CheckInOutController::checkOut()</code>
<code>status</code>	ENUM(<code>confirmed</code> , <code>checked_in</code> , <code>checked_out</code> , <code>cancelled</code> , <code>no_show</code>)	See Section 7.4 lifecycle
<code>total_nights</code>	SMALLINT	<code>check_out_date - check_in_date</code> in days; used to generate room-rent <code>folio_charges</code> on check-in
<code>quoted_rate</code>	DECIMAL(10,2)	Agreed nightly rate; snapshot of the applicable <code>base_price_per_night</code> at booking time
<code>created_by</code>	BIGINT	S2 <code>employees.id</code> reference — the receptionist who created the reservation

room_folios

One folio per reservation. Opened by `CheckInOutController::checkIn()` and settled before `checkOut()` is permitted.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>folio_number</code>	VARCHAR(20), UNIQUE	Auto-generated for the printed invoice (Section 9.3)
<code>reservation_id</code>	BIGINT, FK → <code>reservations.id</code>	
<code>guest_id</code>	BIGINT, FK → <code>guest_profiles.id</code>	Denormalized from the reservation for efficient invoice generation
<code>room_id</code>	BIGINT, FK → <code>rooms.id</code>	Denormalized from the reservation
<code>status</code>	ENUM(<code>open</code> , <code>settled</code> , <code>closed</code>)	<code>settled</code> when <code>outstanding_balance = 0</code> ; <code>closed</code> after post-checkout administrative closure
<code>total_charges</code>	DECIMAL(14,2)	Running total of all <code>folio_charges.net_amount</code> values
<code>total_paid</code>	DECIMAL(14,2)	Running total of all <code>folio_payments.amount</code> values

Column	Type	Notes
<code>outstanding_balance</code>	DECIMAL(14,2)	<code>total_charges - total_paid</code> ; recomputed on every charge or payment
<code>opened_at</code>	TIMESTAMP	
<code>settled_at</code>	TIMESTAMP NULL	Set when <code>outstanding_balance</code> reaches 0

folio_charges

One row per charge posted to a folio — room rent on check-in, F&B via ID5, or any manually posted incidental.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>folio_id</code>	BIGINT, FK → <code>room_folios.id</code>	
<code>service_category</code>	ENUM(<code>room_rent</code> , <code>fnb</code> , <code>laundry</code> , <code>minibar</code> , <code>event</code> , <code>other</code>)	
<code>description</code>	VARCHAR(255)	e.g. "Room 101 — 3 nights", "Restaurant Order #ORD-0042"
<code>subtotal</code>	DECIMAL(12,2)	Pre-tax amount
<code>service_charge_rate</code>	DECIMAL(5,4)	Rate snapshot at charge time
<code>service_charge_amount</code>	DECIMAL(12,2)	<code>round(subtotal × service_charge_rate, 2)</code>
<code>vat_rate</code>	DECIMAL(5,4)	Rate snapshot at charge time
<code>vat_amount</code>	DECIMAL(12,2)	<code>round((subtotal + service_charge_amount) × vat_rate, 2)</code>
<code>net_amount</code>	DECIMAL(12,2)	<code>subtotal + service_charge_amount + vat_amount</code> ; added to <code>room_folios.total_charges</code>
<code>cashier_id</code>	BIGINT	S2 <code>employees.id</code> reference — the receptionist or cashier posting the charge
<code>reference_type</code>	VARCHAR(50) NULL	e.g. <code>restaurant_order</code> , <code>reservation</code> — the source of the charge
<code>reference_id</code>	BIGINT NULL	ID of the source record
<code>charged_at</code>	TIMESTAMP	

NOTE

`FolioService::addCharge()` is called by `CheckInOutController::checkIn()` once per night (iterating `total_nights`) to post `room_rent` charges, and is called in-process by `BillingController::pay()` via `FolioService::postChargeForRoom()` to post `fnb` charges (ID5). Both paths compute SC/VAT using the

same formula as the Restaurant module (Section 5.3) and store rate snapshots on the charge row so the correct rates are preserved even if the configuration changes later.

folio_payments

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>folio_id</code>	BIGINT, FK → <code>room_folios.id</code>	
<code>amount</code>	DECIMAL(12,2)	
<code>payment_method</code>	ENUM(<code>cash</code> , <code>bank_transfer</code> , <code>pos</code> , <code>visa</code> , <code>mobile_banking</code>)	
<code>cashier_id</code>	BIGINT	S2 <code>employees.id</code> reference
<code>shift_id</code>	BIGINT, FK → <code>cashier_shifts.id</code>	Links to the cashier's open shift for reconciliation — unlike <code>bill_payments.shift_id</code> , this is non-nullable (every hotel payment must be tied to an open shift)
<code>paid_at</code>	TIMESTAMP	
<code>reference_number</code>	VARCHAR(80) NULL	

cashier_shifts

New table for S3 — consolidates shift tracking for cashiers who handle both Restaurant and Hotel transactions, so their combined takings reconcile against one shift record.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>cashier_id</code>	BIGINT	S2 <code>employees.id</code> reference — not a real FK (cross-database)
<code>opened_at</code>	TIMESTAMP	
<code>closed_at</code>	TIMESTAMP NULL	Set when the cashier submits the shift for reconciliation
<code>opening_cash_float</code>	DECIMAL(10,2) NULL	Cash float counted at shift open
<code>closing_cash_counted</code>	DECIMAL(10,2) NULL	Cash counted by the cashier at shift close
<code>expected_cash</code>	DECIMAL(10,2) NULL	Computed by <code>CashierReportController::closeShift()</code> from all <code>folio_payments</code> + <code>bill_payments</code> with <code>payment_method = cash</code> for this shift
<code>variance</code>	DECIMAL(10,2) NULL	<code>closing_cash_counted - expected_cash</code> ; negative = short, positive = over
<code>status</code>	ENUM(<code>open</code> , <code>closed</code>)	

3.4 Platform plumbing tables

event_outbox

Same structure as S1 and S2 (D2, ID10). Every cross-system event S3 emits (Section 8.1) is written here first, then published by the `PublishOutboxEvents` job.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>event_name</code>	VARCHAR(100)	e.g. <code>wh.events.s3.folio.settled</code>
<code>payload</code>	JSON	
<code>status</code>	ENUM(<code>pending</code> , <code>published</code> , <code>failed</code>)	
<code>attempts</code>	SMALLINT	
<code>created_at</code> / <code>published_at</code>	TIMESTAMP	

idempotency_keys

Required by D5 because S3 is both a caller (outbound to S2 and S4) and a callee (inbound `PUT /purchase-orders/{id}/approve` , `POST /bills/{id}/payments` , `POST /folios/{id}/payments`) of idempotent endpoints.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>idempotency_key</code>	VARCHAR(80), UNIQUE	Client-supplied <code>Idempotency-Key</code> header value
<code>endpoint</code>	VARCHAR(120)	e.g. <code>PUT /api/v1/purchase-orders/{id}/approve</code>
<code>request_hash</code>	VARCHAR(64)	SHA-256 of the normalised request body; a replay with a different body for the same key is rejected (<code>409 IDEMPOTENCY_KEY_CONFLICT</code>)
<code>response_body</code>	JSON	Cached response, replayed verbatim on retry
<code>status_code</code>	SMALLINT	
<code>created_at</code>	TIMESTAMP	
<code>expires_at</code>	TIMESTAMP	Purged daily at 02:00 by <code>PurgeIdempotencyKeys</code> (D5)

4. API Endpoint Catalog

All endpoints below are prefixed with `/api/v1` and reached through the S1 gateway at `/s3/api/v1/*` (S0 §3). Every request must carry a verified JWT; the **Permission** column lists the `permissions[]` claim required by `CheckPermission` (Section 6 has the full catalog with role grants). Idempotency-Key-required endpoints are marked **(D5)**.

4.1 Inventory & Purchasing module

Method & Path	Permission	Description
GET <code>/item-categories</code>	<code>S3.inventory.items.read</code>	apiResource
POST <code>/item-categories</code>	<code>S3.inventory.items.write</code>	apiResource
PUT <code>/item-categories/{id}</code>	<code>S3.inventory.items.write</code>	apiResource
DELETE <code>/item-categories/{id}</code>	<code>S3.inventory.items.write</code>	Soft delete via <code>is_active = false</code>
GET <code>/items</code>	<code>S3.inventory.items.read</code>	Paginated list; filterable by category, low-stock, active status
POST <code>/items</code>	<code>S3.inventory.items.write</code>	Create catalogue entry
GET <code>/items/{id}</code>	<code>S3.inventory.items.read</code>	Item detail with current stock
PUT <code>/items/{id}</code>	<code>S3.inventory.items.write</code>	Update catalogue entry
GET <code>/items/{id}/stock</code>	<code>S3.inventory.items.read</code>	Current stock + active batch breakdown
GET <code>/items/{id}/movements</code>	<code>S3.inventory.items.read</code>	Movement history, paginated
POST <code>/stock/adjustments</code>	<code>S3.inventory.stock.write</code>	Manual stock adjustment; writes a <code>stock_movements</code> row (<code>movement_type = adjustment</code>)
POST <code>/stock/write-offs</code>	<code>S3.inventory.stock.write</code>	Write-off; sets batch <code>status = expired</code> where applicable
GET <code>/stock/expiry-alerts</code>	<code>S3.inventory.items.read</code>	Batches expiring within <code>EXPIRY_ALERT_DAYS</code>
GET <code>/stock/low-stock-alerts</code>	<code>S3.inventory.items.read</code>	Items below <code>minimum_stock_level</code>
GET <code>/stock/valuation</code>	<code>S3.inventory.reports.read</code>	Inventory valuation report (current stock × batch unit cost, FIFO/FEFO weighted)
GET <code>/suppliers</code>	<code>S3.inventory.suppliers.read</code>	Paginated list
POST <code>/suppliers</code>	<code>S3.inventory.suppliers.write</code>	Create supplier
GET <code>/suppliers/{id}</code>	<code>S3.inventory.suppliers.read</code>	Supplier detail with outstanding balance
PUT <code>/suppliers/{id}</code>	<code>S3.inventory.suppliers.write</code>	Update supplier
POST <code>/suppliers/{id}/payments</code>	<code>S3.inventory.suppliers.write</code>	Record supplier payment; Idempotency-Key required (D5)

Method & Path	Permission	Description
GET /purchase-orders	S3.inventory.purchase_orders.read	Paginated list; filterable by status, supplier
POST /purchase-orders	S3.inventory.purchase_orders.write	Create PO (status = <code>draft</code>)
GET /purchase-orders/{id}	S3.inventory.purchase_orders.read	PO detail with lines
PUT /purchase-orders/{id}/submit	S3.inventory.purchase_orders.write	<code>draft</code> → <code>pending_dept_head</code>
PUT /purchase-orders/{id}/approve	S3.inventory.purchase_orders.approve	Role-based tier approval (Section 5.3); Idempotency-Key required (D5)
PUT /purchase-orders/{id}/cancel	S3.inventory.purchase_orders.write	Cancel from any pending state
POST /purchase-orders/{id}/goods-receipts	S3.inventory.stock.write	Record a goods receipt against an approved PO
GET /goods-receipts/{id}	S3.inventory.stock.read	Receipt detail with lines

CONVENTION

Removed endpoints (ID4). The original M5 spec exposed `POST /items/dispatch` as a dedicated endpoint so M6 could call it over HTTP on order finalization. In S3, `FifoFefoService::dispatch()` is called directly in-process — **this endpoint does not exist in S3** and has no S3 equivalent. No external caller should attempt to drive ingredient dispatch via an HTTP endpoint.

4.2 Restaurant & F&B module

Method & Path	Permission	Description
GET /menu-categories	S3.restaurant.menu.read	apiResource
POST /menu-categories	S3.restaurant.menu.write	apiResource
PUT /menu-categories/{id}	S3.restaurant.menu.write	apiResource
GET /menu-items	S3.restaurant.menu.read	Paginated list; filterable by category, availability
POST /menu-items	S3.restaurant.menu.write	Create menu item
GET /menu-items/{id}	S3.restaurant.menu.read	Menu item detail with recipe ingredients
PUT /menu-items/{id}	S3.restaurant.menu.write	Update menu item
PUT /menu-items/{id}/availability	S3.restaurant.menu.write	Toggle <code>is_available</code>
GET /menu-items/{id}/recipe	S3.restaurant.menu.read	Recipe ingredient list
PUT /menu-items/{id}/recipe	S3.restaurant.menu.write	Replace recipe ingredients
GET /dining-tables	S3.restaurant.orders.read	apiResource
POST /dining-tables	S3.restaurant.menu.write	apiResource
PUT /dining-tables/{id}	S3.restaurant.menu.write	apiResource

Method & Path	Permission	Description
GET /orders	S3.restaurant.orders.read	Paginated list; filterable by status, date, cashier
POST /orders	S3.restaurant.orders.write	Open a new order
GET /orders/{id}	S3.restaurant.orders.read	Order detail with items and bill
POST /orders/{id}/items	S3.restaurant.orders.write	Add item(s) to an open order
DELETE /orders/{id}/items/{itemId}	S3.restaurant.orders.write	Remove item from an open order
POST /orders/{id}/finalize	S3.restaurant.orders.write	Trigger ingredient dispatch (ID4) + bill computation
POST /orders/{id}/cancel	S3.restaurant.orders.write	Cancel an <code>open</code> order
GET /bills/{id}	S3.restaurant.orders.read	Bill detail
POST /bills/{id}/payments	S3.restaurant.billing.write	Record payment; triggers routing (Section 5.4); Idempotency-Key required (D5)
GET /employee-consumption	S3.restaurant.consumption.read	List <code>employee_consumption</code> rows; filterable by period, employee
POST /employee-consumption/push	S3.restaurant.consumption.write	Manually trigger period-close push to S2 (ID12) for the current period

4.3 Hotel & Room Management module

Method & Path	Permission	Description
GET /room-types	S3.hotel.rooms.read	apiResource
POST /room-types	S3.hotel.rooms.write	apiResource
PUT /room-types/{id}	S3.hotel.rooms.write	apiResource
GET /rooms	S3.hotel.rooms.read	Paginated list; filterable by status, type, floor
POST /rooms	S3.hotel.rooms.write	Create room
GET /rooms/{id}	S3.hotel.rooms.read	Room detail with current reservation
PUT /rooms/{id}	S3.hotel.rooms.write	Update room metadata
PUT /rooms/{id}/status	S3.hotel.rooms.write	Set <code>available</code> / <code>maintenance</code> / <code>cleaning</code> (not <code>occupied</code> — that is set by check-in)
GET /guest-profiles	S3.hotel.guests.read	Paginated list
POST /guest-profiles	S3.hotel.guests.write	Create guest profile
GET /guest-profiles/{id}	S3.hotel.guests.read	Guest detail with reservation history
PUT /guest-profiles/{id}	S3.hotel.guests.write	Update guest profile
GET /reservations	S3.hotel.reservations.read	Paginated list; filterable by status, date range, room
POST /reservations	S3.hotel.reservations.write	

Method & Path	Permission	Description
		Create reservation (status = <code>confirmed</code>)
<code>GET /reservations/{id}</code>	<code>S3.hotel.reservations.read</code>	Reservation detail
<code>PUT /reservations/{id}</code>	<code>S3.hotel.reservations.write</code>	Update dates or quoted rate before check-in
<code>PUT /reservations/{id}/cancel</code>	<code>S3.hotel.reservations.write</code>	Cancel a <code>confirmed</code> reservation
<code>PUT /reservations/{id}/no-show</code>	<code>S3.hotel.reservations.write</code>	Mark as <code>no_show</code>
<code>POST /reservations/{id}/check-in</code>	<code>S3.hotel.checkinout.write</code>	Check in; creates <code>room_folios</code> and posts room-rent <code>folio_charges</code>
<code>POST /reservations/{id}/check-out</code>	<code>S3.hotel.checkinout.write</code>	Check out; requires <code>outstanding_balance <= 0</code> (422 otherwise)
<code>GET /folios</code>	<code>S3.hotel.folios.read</code>	Paginated list
<code>GET /folios/{id}</code>	<code>S3.hotel.folios.read</code>	Folio detail with charges and payments
<code>POST /folios/{id}/charges</code>	<code>S3.hotel.folios.write</code>	Manually post an incidental charge (laundry, minibar, etc.)
<code>POST /folios/{id}/payments</code>	<code>S3.hotel.folios.write</code>	Record a folio payment; Idempotency-Key required (D5)
<code>GET /folios/{id}/invoice</code>	<code>S3.hotel.folios.read</code>	Generate the 13-field folio invoice (Section 9.3)
<code>GET /cashier-shifts</code>	<code>S3.hotel.cashier.read</code>	List shifts; filterable by cashier, date
<code>POST /cashier-shifts</code>	<code>S3.hotel.cashier.write</code>	Open a new shift
<code>POST /cashier-shifts/{id}/close</code>	<code>S3.hotel.cashier.write</code>	Close shift; computes <code>expected_cash</code> and <code>variance</code>
<code>GET /cashier-shifts/{id}/report</code>	<code>S3.hotel.cashier.read</code>	Shift reconciliation report (Section 9.4)

CONVENTION

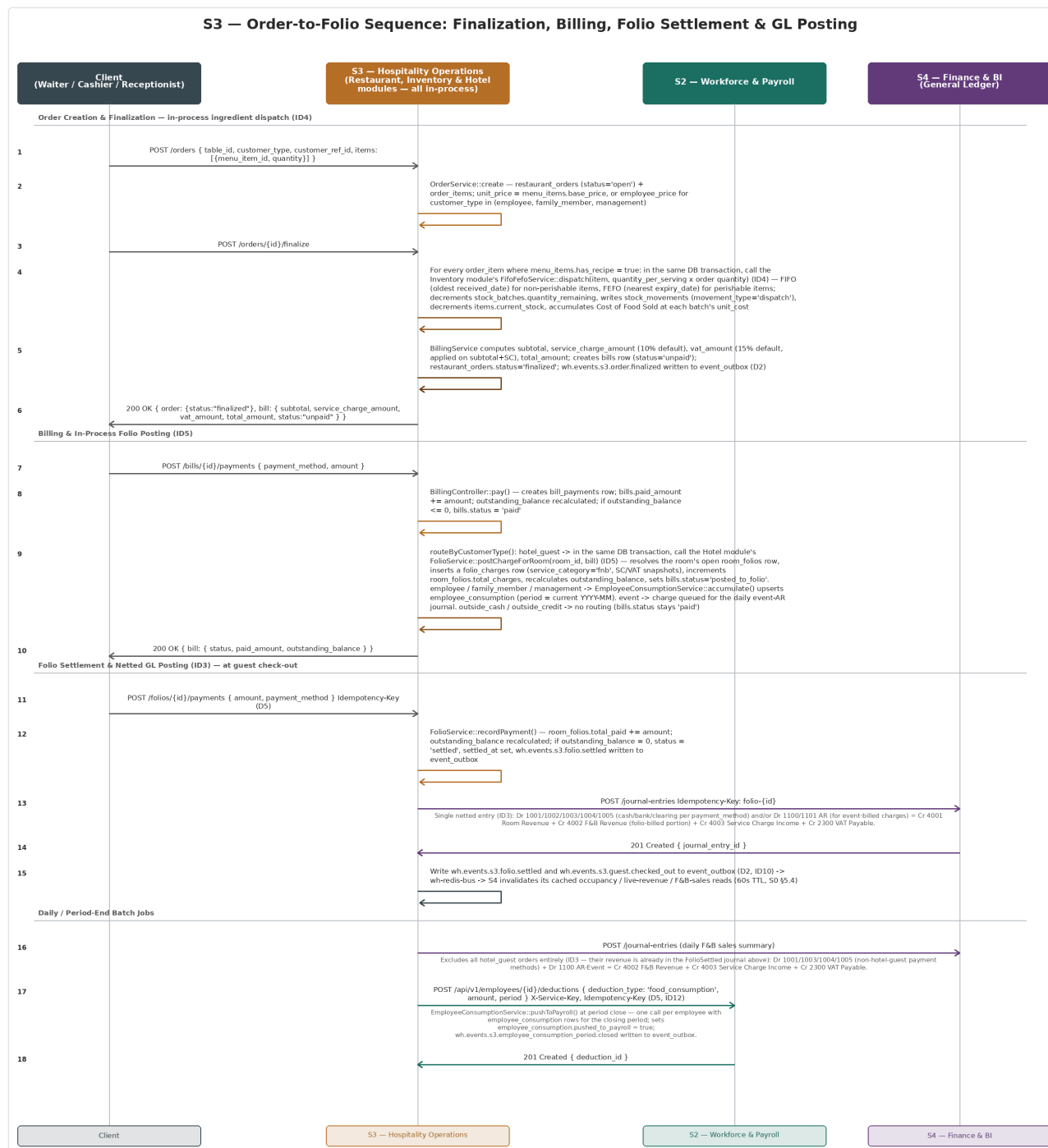
Removed endpoints (ID5). The original M7 spec exposed `GET /rooms/{id}/active-folio` and `POST /folios/{id}/fnb-charge` so M6 could post F&B charges to a room folio over HTTP. In S3, `FolioService::postChargeForRoom()` is called directly in-process by `BillingController::pay()` — **these two endpoints do not exist in S3.** `POST /folios/{id}/charges` is for *manual* incidentals only; automated F&B routing never calls it.

Pagination, errors, rate limits. All list endpoints use `?page= / ?per_page=` cursor-free pagination (D6); error responses follow the standard `{ "error": { "code", "message", "details" } }` envelope (D7); all endpoints are subject to the platform-wide rate limit defined in S0 §10 unless otherwise noted.

5. Core Business Logic

This section describes S3's key calculation engines and process flows in prose and tables. No PHP class code is shown; the descriptions are precise enough for a developer to implement directly in `app/Services/Inventory/*`, `app/Services/Restaurant/*`, and `app/Services/Hotel/*`.

5.1 Order-to-folio sequence overview



Sequence diagram showing 18 numbered steps across four actors — Client (Waiter/Cashier/Receptionist), S3 Hospitality Operations (Restaurant, Inventory, and Hotel modules, all in-process), S2 Workforce & Payroll, and S4 Finance & BI — grouped under four dividers: (1) Order Creation and Finalization: Client creates and finalizes the order; S3 runs `FifoFefoService::dispatch()` in-process (ID4) to decrement stock batches by FIFO/FEFO, then computes SC/VAT bill (status=unpaid). (2) Billing and In-Process Folio Posting: Client pays the bill; S3 calls `routeByCustomerType()` — `hotel_guest` triggers `FolioService::postChargeForRoom()` in-process (ID5) inserting a `folio_charges` row. (3) Folio Settlement and Netted GL Posting: Client posts folio payment; S3 calls S4 `POST /journal-entries` with a single netted room+F&B journal (ID3). (4) Daily/Period-End Batch Jobs: S3 sends daily F&B summary journal to S4 (excluding `hotel_guest` orders already in ID3) and pushes staff consumption deductions to S2 via `POST /employees/{id}/deductions` (ID12).

The numbered steps above correspond to the sequence diagram. Section 5.2–5.8 expand the most complex steps.

5.2 FIFO / FEFO stock rotation (ID4)

`FifoFefoService::dispatch($item, $quantity)` runs inside `OrderController::finalize()`'s database transaction. Its logic:

1. Query `stock_batches` where `item_id = $item->id`, `status = active`, and `quantity_remaining > 0`.
2. Order by `expiry_date ASC` if `$item->rotation_strategy = fefo` (nearest expiry first — used for perishable items: food, dairy, beverages, meat); order by `received_date ASC` if `fifo` (oldest received batch first — used for non-perishables: linen, stationery, cleaning supplies).
3. Iterate the ordered batch set, taking `min(batch.quantity_remaining, remaining_needed)` from each batch:
 - Decrement `stock_batches.quantity_remaining` by the taken amount.
 - If `quantity_remaining` reaches 0, set `stock_batches.status = depleted`.
 - Accumulate Cost of Food Sold at `batch.unit_cost × quantity_taken`.
 - Write a `stock_movements` row: `movement_type = dispatch`, `reference_type = restaurant_order`, `reference_id = $order->id`.
 - Decrement `items.current_stock` by the taken amount.
4. If the iteration exhausts all active batches before satisfying `$quantity`, throw `InsufficientStockException` — the entire `finalize()` transaction rolls back.
5. Return the list of `[batch_id, quantity_taken, unit_cost]` tuples to the caller so the COGS journal entry can be computed.

Strategy	Item types	Sort column	Sort direction
<code>fifo</code>	Non-perishables (linen, stationery, cleaning supplies)	<code>received_date</code>	ASC (oldest first)
<code>fefo</code>	Perishables (food, dairy, beverages, meat)	<code>expiry_date</code>	ASC (nearest expiry first)

5.3 PO approval thresholds

`PurchaseOrderController::approve()` checks the acting user's role against the current `purchase_orders.status` and `total_amount` before advancing the status:

PO <code>total_amount</code> (ETB)	Required approvers (in order)	Status progression
Less than 5,000	Department Head only	<code>pending_dept_head</code> → <code>approved</code>
5,000 to 49,999	Department Head, then Finance Manager	<code>pending_dept_head</code> → <code>pending_finance</code> → <code>approved</code>
50,000 and above	Department Head, Finance Manager, General Manager	<code>pending_dept_head</code> → <code>pending_finance</code> → <code>pending_gm</code> → <code>approved</code>

When a PO reaches `approved`, `PurchaseOrderController::approve()` also:

- Sets `purchase_orders.approved_by` and `approved_at`.
- Writes `wh.events.s3.purchase_order.approved` to `event_outbox` with `approval_tier` = 1, 2, or 3 depending on the `total_amount` tier.
- Posts the AP journal entry to S4 via `FinanceClientService`: `Dr 1200/1201 Inventory (per line) / Cr 2001 AP – Suppliers` (S0 §8).

The two `.env` thresholds (`PO_DEPT_HEAD_THRESHOLD` , `PO_FINANCE_THRESHOLD`) must match the ETB values in the table above. If the values are changed, existing POs in progress retain their current approval path — the tier is determined at submission time.

5.4 Service charge and VAT formula

Every bill (Restaurant module) and every folio charge (Hotel module) applies SC and VAT in the following order. Rate snapshots are stored on every `bills` and `folio_charges` row at creation time.

```
service_charge_amount = round(subtotal × service_charge_rate, 2)
vat_amount = round((subtotal + service_charge_amount) × vat_rate, 2)
total_amount = subtotal + service_charge_amount + vat_amount
```

Default rates: `service_charge_rate = 0.10` (10%), `vat_rate = 0.15` (15%). These are configured via `.env` (`SC_RATE`, `VAT_RATE`) so the rates can be adjusted without a code change; the snapshot columns on `bills` and `folio_charges` preserve the rate that applied at the time each charge was created.

CONVENTION

Mandatory SC and VAT. Service charge and VAT apply to **all** F&B bills and **all** folio charges, for every customer type — including employee meals and hotel-guest room-rent charges. There are no exemptions within the system. If a future regulatory change requires an exemption for a specific customer type, that must be implemented as a dedicated configuration flag, not by hardcoding a zero rate for a category.

5.5 Customer types and post-payment billing routing

`restaurant_orders.customer_type` is set when the order is opened and drives two decisions: which price to use (`base_price` vs `employee_price`) and what routing to apply after the bill is fully paid.

Customer Type	Price Used	Post-Payment Routing
<code>outside_cash</code>	<code>base_price</code>	None — bill stays <code>paid</code> , direct cash/ card collection
<code>outside_credit</code>	<code>base_price</code>	None — bill stays <code>paid</code> , collected against a pre-arranged credit account
<code>hotel_guest</code>	<code>base_price</code>	ID5 — <code>FolioService::postChargeForRoom()</code> inserts a <code>folio_charges</code> row into the room folio; <code>bills.status</code> → <code>posted_to_folio</code>
<code>employee</code>	<code>employee_price</code> (or <code>base_price</code> if NULL)	<code>EmployeeConsumptionService::accumulate()</code> upserts <code>employee_consumption</code> ; period-end push to S2 (ID12)
<code>family_member</code>	<code>employee_price</code> (or <code>base_price</code> if NULL)	Same as <code>employee</code> — treated identically for consumption tracking
<code>management</code>	<code>employee_price</code> (or <code>base_price</code> if NULL)	Same as <code>employee</code>
<code>event</code>	<code>base_price</code>	Charge queued for the daily event-AR journal to S4 (Dr 1101 AR-Events / Cr 4004 Event Revenue + SC/VAT)

`routeByCustomerType()` is called by `BillingController::pay()` inside the same transaction as the `bill_payments` row creation. For `hotel_guest`, it calls `FolioService::postChargeForRoom($room_id,`

`$bill`) directly. For `employee / family_member / management`, it calls `EmployeeConsumptionService::accumulate($employee_id, $period, $bill->subtotal)` using an upsert on `(employee_id, period)`. For `event`, it posts a synchronous journal entry to S4 or queues it for the daily batch (implementation choice — the end result must be `Dr 1101 AR-Events = Cr 4004 Event Revenue + Cr 4003 SC + Cr 2300 VAT`).

5.6 Folio charges and check-in / check-out logic

Check-in (`CheckInOutController::checkIn()`):

1. Validate that the reservation `status = confirmed` and the target room `status = available`.
2. Create a `room_folios` row (`status = open`, `total_charges = 0`, `total_paid = 0`, `outstanding_balance = 0`).
3. For each night (1 to `reservations.total_nights`), call `FolioService::addCharge($folio, room_rent, $quoted_rate)`, which computes SC/VAT and creates a `folio_charges` row, incrementing `room_folios.total_charges` and `outstanding_balance`.
4. Set `rooms.status = occupied`, `rooms.current_reservation_id = reservation.id`.
5. Set `reservations.status = checked_in`, `actual_check_in = now()`.
6. Write `wh.events.s3.guest.checked_in` to `event_outbox`.

Check-out (`CheckInOutController::checkOut()`):

1. Validate that `room_folios.outstanding_balance <= 0`; if not, return `422 UNPROCESSABLE` (D7).
2. Set `reservations.status = checked_out`, `actual_check_out = now()`.
3. Set `rooms.status = cleaning`, `rooms.current_reservation_id = NULL`.
4. Write `wh.events.s3.guest.checked_out` to `event_outbox`.

NOTE

The `outstanding_balance <= 0` guard on check-out enforces that the folio has been settled (`FolioService::recordPayment()` already set `room_folios.status = settled`) before the guest can leave. If a guest has outstanding charges and the hotel chooses to post them to a credit account, the receptionist must post a folio payment with `payment_method` indicating the credit arrangement to clear the balance before `checkOut()` will succeed.

5.7 Partial folio payment management

A folio can receive multiple payments across any number of calls to `POST /folios/{id}/payments`. Each call must carry an `Idempotency-Key` header (D5). `FolioService::recordPayment()`:

1. Creates a `folio_payments` row with the provided `amount`, `payment_method`, `cashier_id`, and `shift_id`.
2. Increments `room_folios.total_paid` by `amount`.
3. Recomputes `outstanding_balance = total_charges - total_paid`.
4. If `outstanding_balance = 0`, sets `room_folios.status = settled`, `settled_at = now()`, and:
 - Posts the netted folio-settlement journal to S4 (step 13 in Section 5.1, S0 §8.1).
 - Writes `wh.events.s3.folio.settled` to `event_outbox`.

Step	Action	Trigger
1	Receptionist posts partial payment	<code>POST /folios/{id}/payments</code>
2	<code>FolioService::recordPayment()</code> creates <code>folio_payments</code> row	Same transaction

Step	Action	Trigger
3	<code>room_folios.total_paid</code> incremented; <code>outstanding_balance</code> recomputed	Same transaction
4	<code>outstanding_balance > 0</code> — folio remains <code>open</code>	No further action
5	Receptionist posts additional payment(s) until balance reaches 0	Repeat <code>POST /folios/{id}/payments</code>
6	<code>outstanding_balance = 0</code> — folio <code>settled</code> ; netted journal posted to S4	<code>FolioService::recordPayment()</code>

5.8 Employee F&B consumption push to payroll (ID12)

At the end of each billing period (typically month-end), `EmployeeConsumptionService::pushToPayroll()` is triggered either manually via `POST /employee-consumption/push` or by a scheduled command. For each `employee_consumption` row in the closing period where `pushed_to_payroll = false`:

1. Call S2's `POST /api/v1/employees/{employee_id}/deductions` with `{ "deduction_type": "food_consumption", "amount": total_amount, "period": "YYYY-MM" }`.
2. Include `X-Service-Key` header (D9) and `Idempotency-Key` header (D5) — the key is `s3-meal-{employee_id}-{period}`.
3. On `201 Created`, set `employee_consumption.pushed_to_payroll = true`.
4. After all employees are processed, write `wh.events.s3.employee_consumption_period.closed` to `event_outbox`.

If the S2 call returns a non-2xx response, the row is left with `pushed_to_payroll = false` for retry. Because the `Idempotency-Key` is deterministic (`s3-meal-{employee_id}-{period}`), a retry for the same employee in the same period is always safe — S2 will return the cached response from the first successful call.

6. Role-Based Access Control

S3 enforces authorization with the same `CheckPermission` middleware described in S0 §4 — it reads the `permissions[]` claim from the verified JWT and rejects the request with `403` if the required permission is absent. This section catalogs every S3 permission, grouped by module per ID9's `S3.{domain}.{resource}.{action}` convention.

6.1 Permission catalog

Inventory domain (`S3.inventory.*`)

Permission	Default Roles
<code>S3.inventory.items.read</code>	<code>inventory_manager</code> , <code>restaurant_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.inventory.items.write</code>	<code>inventory_manager</code> , <code>super_admin</code>
<code>S3.inventory.stock.read</code>	<code>inventory_manager</code> , <code>restaurant_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.inventory.stock.write</code>	<code>inventory_manager</code> , <code>super_admin</code>
<code>S3.inventory.suppliers.read</code>	<code>inventory_manager</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.inventory.suppliers.write</code>	<code>inventory_manager</code> , <code>super_admin</code>
<code>S3.inventory.purchase_orders.read</code>	<code>inventory_manager</code> , <code>department_head</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.inventory.purchase_orders.write</code>	<code>inventory_manager</code> , <code>department_head</code> , <code>super_admin</code>
<code>S3.inventory.purchase_orders.approve</code>	<code>department_head</code> (tier 1), <code>finance_manager</code> (tier 2), <code>general_manager</code> (tier 3), <code>super_admin</code>
<code>S3.inventory.reports.read</code>	<code>inventory_manager</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>

Restaurant domain (`S3.restaurant.*`)

Permission	Default Roles
<code>S3.restaurant.menu.read</code>	<code>restaurant_manager</code> , <code>cashier</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.restaurant.menu.write</code>	<code>restaurant_manager</code> , <code>super_admin</code>
<code>S3.restaurant.orders.read</code>	<code>restaurant_manager</code> , <code>cashier</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.restaurant.orders.write</code>	<code>restaurant_manager</code> , <code>cashier</code> , <code>super_admin</code>
<code>S3.restaurant.billing.write</code>	<code>restaurant_manager</code> , <code>cashier</code> , <code>super_admin</code>
<code>S3.restaurant.consumption.read</code>	<code>restaurant_manager</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.restaurant.consumption.write</code>	<code>restaurant_manager</code> , <code>super_admin</code>

Hotel domain (`S3.hotel.*`)

Permission	Default Roles
<code>S3.hotel.rooms.read</code>	<code>receptionist</code> , <code>cashier</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.hotel.rooms.write</code>	<code>receptionist</code> , <code>super_admin</code>
<code>S3.hotel.guests.read</code>	<code>receptionist</code> , <code>cashier</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.hotel.guests.write</code>	<code>receptionist</code> , <code>super_admin</code>
<code>S3.hotel.reservations.read</code>	<code>receptionist</code> , <code>cashier</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.hotel.reservations.write</code>	<code>receptionist</code> , <code>super_admin</code>
<code>S3.hotel.checkinout.write</code>	<code>receptionist</code> , <code>super_admin</code>
<code>S3.hotel.folios.read</code>	<code>receptionist</code> , <code>cashier</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.hotel.folios.write</code>	<code>receptionist</code> , <code>cashier</code> , <code>super_admin</code>
<code>S3.hotel.cashier.read</code>	<code>receptionist</code> , <code>cashier</code> , <code>finance_manager</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S3.hotel.cashier.write</code>	<code>cashier</code> , <code>receptionist</code> , <code>super_admin</code>

6.2 System roles — S3 access summary

This table mirrors S0 §7.2's Role → System Access Matrix, filtered to S3:

Role	S3 Access
<code>inventory_manager</code>	Full — all <code>S3.inventory.*</code> permissions
<code>restaurant_manager</code>	Full — all <code>S3.restaurant.*</code> permissions; read access to <code>S3.inventory.items.read</code> and <code>S3.inventory.stock.read</code> (needed for recipe management and low-stock visibility)
<code>receptionist</code>	Full — all <code>S3.hotel.*</code> permissions
<code>cashier</code>	Transactions — <code>S3.restaurant.orders.write</code> , <code>S3.restaurant.billing.write</code> , <code>S3.hotel.folios.write</code> , <code>S3.hotel.cashier.write</code> ; read access to menus, orders, folios, and rooms
<code>department_head</code>	PO approval (tier 1) — <code>S3.inventory.purchase_orders.approve</code> , <code>S3.inventory.purchase_orders.read</code>
<code>finance_manager</code>	PO approval (tier 2) — <code>S3.inventory.purchase_orders.approve</code> , <code>S3.inventory.purchase_orders.read</code> , <code>S3.inventory.suppliers.read</code> , <code>S3.inventory.reports.read</code> , <code>S3.hotel.folios.read</code> , <code>S3.hotel.cashier.read</code> , <code>S3.restaurant.consumption.read</code>

Role	S3 Access
<code>general_manager</code>	PO approval (tier 3) + Read — all <code>*.read</code> permissions across Inventory, Restaurant, and Hotel modules
<code>report_viewer</code>	Read — all <code>*.read</code> permissions across the three modules
<code>super_admin</code>	Full — every permission listed above
<code>hr_manager</code>	— (no S3 access; S3 holds no employee master data)
<code>payroll_officer</code>	— (no S3 access; payroll deductions arrive as inbound calls from S3 to S2, not the reverse)
<code>accountant</code>	— (no S3 access; financial data is consumed via S4's reporting layer)

DESIGN DECISION

Permission renaming (ID9). The original catalogs used `M5.*`, `M6.*`, `M7.*` (e.g. `M5.inventory.write`, `M6.billing.pay`, `M7.folio.settle`). Under the unified `S{n}.{domain}.{resource}.{action}` convention, these become `S3.inventory.*`, `S3.restaurant.*`, and `S3.hotel.*` as catalogued above. Anywhere a former M5/M6/M7 permission string appears in legacy documentation or seed files, treat it as superseded by its S3 equivalent in this catalog.

PO approval permissions across roles. `S3.inventory.purchase_orders.approve` is granted to `department_head`, `finance_manager`, and `general_manager`, but `PurchaseOrderController::approve()` additionally checks the **current status** against the **acting role** — a `department_head` can only approve from `pending_dept_head`, a `finance_manager` only from `pending_finance`, and a `general_manager` only from `pending_gm`. Having the permission alone is not sufficient; the permission check and the status check must both pass.

6.3 CheckPermission middleware contract

`CheckPermission` behaves identically to the contract described in S0 §4 and implemented in S1 and S2:

- Reads `permissions[]` from the verified JWT (already validated by `VerifyJwt` upstream at the gateway).
- Route definitions in `routes/api.php` declare the required permission via middleware parameter, e.g. `>middleware('permission:S3.hotel.folios.write')`.
- Missing permission → `403 Forbidden` with the standard error envelope (D7).
- The PO approval tier check is an **additional** status check inside `PurchaseOrderController::approve()`, separate from `CheckPermission` middleware.

6.4 Permission seeding

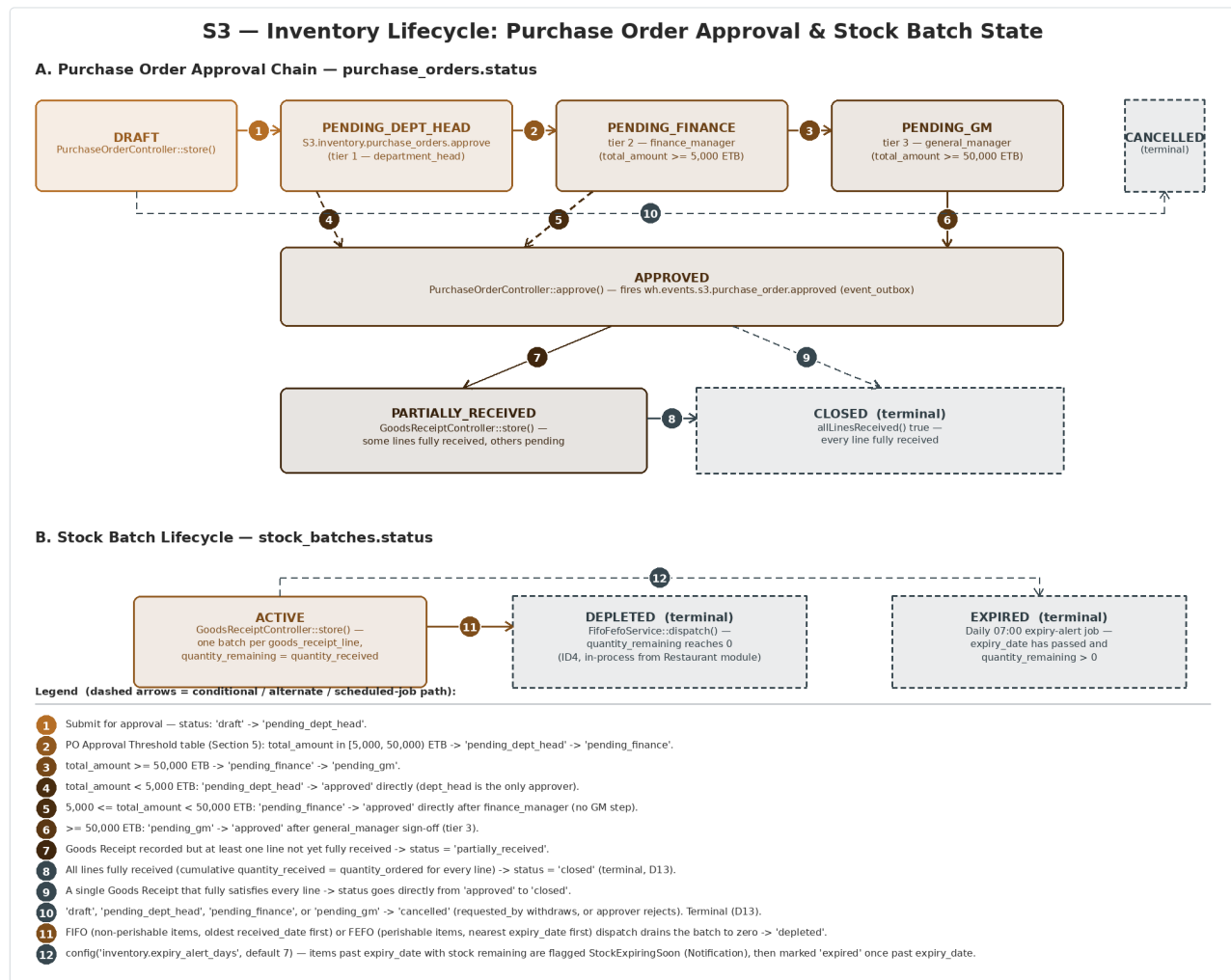
CROSS-REFERENCE

The master list of all platform permissions — including every `S3.*` permission in this catalog — is seeded into S1's `permissions` table and assigned to roles via S1's `role_permissions` table, as described in **S0 §7.1** and **S1 §6.4 (Permission Seeding)**. S3 does not store permissions or roles itself; it only reads the `permissions[]` claim from the JWT.

7. Inventory & Hospitality Lifecycle

S3 manages four independent state machines: the **purchase order approval chain** (`purchase_orders.status`), the **stock batch lifecycle** (`stock_batches.status`), the **restaurant order and bill lifecycle** (`restaurant_orders.status` + `bills.status`), and the **reservation, room, and folio lifecycle** (`reservations.status`, `rooms.status`, `room_folios.status`). The two diagrams below show these state machines.

7.1 Section A — Purchase order approval chain (`purchase_orders.status`)



State diagram with two sections. Section A — Purchase Order Approval Chain: states `DRAFT` → `PENDING_DEPT_HEAD` → `PENDING_FINANCE` → `PENDING_GM` → `APPROVED` → `PARTIALLY_RECEIVED` → `CLOSED` (terminal); dashed shortcuts bypass `PENDING_FINANCE` for amounts under 5,000 ETB and bypass `PENDING_GM` for amounts in the 5,000–49,999 ETB range; `CANCELLED` (terminal) reachable from any pending state. Section B — Stock Batch Lifecycle: state `ACTIVE` transitions to `DEPLETED` (terminal) when `FifoFefoService::dispatch()` drains `quantity_remaining` to zero (transition 11), and to `EXPIRED` (terminal) when the daily 07:00 expiry-alert job finds `expiry_date` past with stock still remaining (transition 12, dashed).

State	Meaning	Entered via
<code>draft</code>	Created but not yet submitted	(1) <code>POST /purchase-orders</code>
<code>pending_dept_head</code>		

State	Meaning	Entered via
	Awaiting department head approval (tier 1)	(1) <code>PUT /purchase-orders/{id}/submit</code>
<code>pending_finance</code>	Awaiting finance manager approval (tier 2)	(2) After dept head approves a PO in the 5,000–49,999 ETB range
<code>pending_gm</code>	Awaiting general manager approval (tier 3)	(3) After finance manager approves a PO $\geq 50,000$ ETB
<code>approved</code>	Fully approved; goods can now be received	(4) $<5k$ ETB: dept head only; (5) $5k-49.9k$: after finance manager; (6) $\geq 50k$: after GM
<code>partially_received</code>	At least one goods receipt recorded but not all lines fully received	(7) <code>POST /purchase-orders/{id}/goods-receipts</code>
<code>closed</code>	All lines fully received — terminal	(8) Last line fully received; (9) Single full-receipt delivery
<code>cancelled</code>	Withdrawn before delivery — terminal	(10) <code>PUT /purchase-orders/{id}/cancel</code> from any pending state

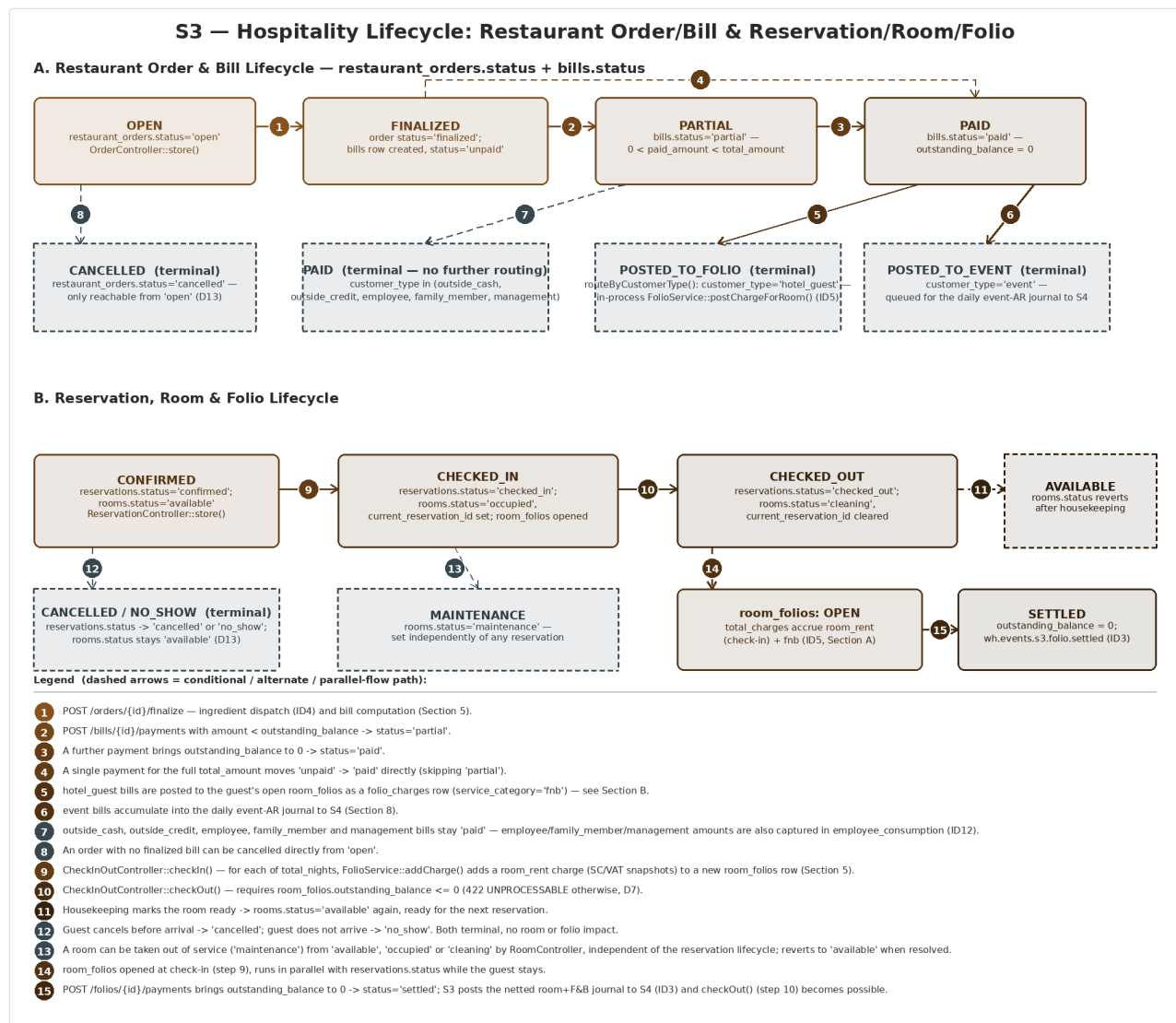
CONVENTION

D13 — closed and cancelled POs are immutable. Once `purchase_orders.status` reaches `closed` or `cancelled`, no lines may be added, modified, or deleted. The PO is part of the procurement audit trail connecting to goods receipts, stock batches, and AP journal entries already posted to S4.

7.2 Section B — Stock batch lifecycle (`stock_batches.status`)

State	Meaning	Entered via
<code>active</code>	In stock, available for dispatch	(11-default) Created by <code>GoodsReceiptController::store()</code> with <code>quantity_remaining = quantity_received</code>
<code>depleted</code>	Fully consumed — terminal	(11) <code>FifoFefoService::dispatch()</code> reduces <code>quantity_remaining</code> to 0
<code>expired</code>	Passed expiry date with stock remaining — terminal	(12) Daily 07:00 scheduled command (<code>RaiseExpiryAlerts</code>) — marks batches where <code>expiry_date < today</code> and <code>quantity_remaining > 0</code> ; fires <code>StockExpiringSoon</code> notification to <code>inventory_manager</code> and <code>super_admin</code> for batches within <code>EXPIRY_ALERT_DAYS</code> days before they actually expire

7.3 Section C — Restaurant order and bill lifecycle



State diagram with two sections. Section A — Restaurant Order/Bill Lifecycle: restaurant_orders states OPEN → FINALIZED (via finalize with ID4 ingredient dispatch and bill computation) → CANCELLED (terminal, only from OPEN); bills states UNPAID → PARTIAL → PAID → POSTED_TO_FOLIO (terminal for hotel_guest via FolioService::postChargeForRoom(), ID5) or POSTED_TO_EVENT (terminal for event orders). Section B — Reservation/Room/Folio Lifecycle: reservation states CONFIRMED → CHECKED_IN → CHECKED_OUT → CANCELLED/NO_SHOW (terminal); room states AVAILABLE → OCCUPIED → CLEANING → AVAILABLE, with MAINTENANCE branching independently; room_folios OPEN (created at check-in) → SETTLED when outstanding_balance reaches zero, triggering the netted folio journal to S4 (ID3).

State	Meaning	Entered via
open	Order accepted, items can be added	(1-default) POST /orders
finalized	Bill computed, no more items can be added	(1) POST /orders/{id}/finalize
cancelled	Order voided — terminal; only from open	(8) POST /orders/{id}/cancel

bills.status advances independently once the bills row is created on finalization:

State	Meaning	Entered via
<code>unpaid</code>	Bill created; no payments yet	Created by <code>BillingService</code> on finalize
<code>partial</code>	At least one payment received; <code>outstanding_balance > 0</code>	(2) <code>POST /bills/{id}/payments</code> with <code>amount < outstanding_balance</code>
<code>paid</code>	<code>outstanding_balance = 0</code>	(3) Further payment; or (4) single full payment directly from <code>unpaid</code>
<code>posted_to_folio</code>	Charge posted to room folio — terminal	(5) <code>routeByCustomerType()</code> for <code>hotel_guest</code> (ID5)
<code>posted_to_event</code>	Charge posted to event AR — terminal	(6) <code>routeByCustomerType()</code> for <code>event</code>

7.4 Section D — Reservation, room & folio lifecycle

Resource	State	Meaning	Entered via
<code>reservations</code>	<code>confirmed</code>	Booked, guest not yet arrived	(default) <code>POST /reservations</code>
<code>reservations</code>	<code>checked_in</code>	Guest in-house, folio open	(9) <code>CheckInOutController::checkIn()</code>
<code>reservations</code>	<code>checked_out</code>	Guest departed, folio settled	(10) <code>CheckInOutController::checkOut()</code>
<code>reservations</code>	<code>cancelled</code>	Cancelled before arrival — terminal	(12) <code>PUT /reservations/{id}/cancel</code>
<code>reservations</code>	<code>no_show</code>	Guest did not arrive — terminal	(12) <code>PUT /reservations/{id}/no-show</code>
<code>rooms</code>	<code>available</code>	Ready for new reservation	Default; restored after housekeeping (11)
<code>rooms</code>	<code>occupied</code>	Guest checked in	(9) <code>checkIn()</code>
<code>rooms</code>	<code>cleaning</code>	Awaiting housekeeping after checkout	(10) <code>checkOut()</code>
<code>rooms</code>	<code>maintenance</code>	Out of service — independent of reservation flow	(13) <code>PUT /rooms/{id}/status</code>
<code>room_folios</code>	<code>open</code>	Folio accumulating charges	(14) Created by <code>checkIn()</code>
<code>room_folios</code>	<code>settled</code>	<code>outstanding_balance = 0</code> ; netted journal posted to S4	(15) <code>FolioService::recordPayment()</code>
<code>room_folios</code>	<code>closed</code>	Administrative closure post-checkout	Manual via <code>PUT /folios/{id}</code>

8. Event-Driven Integration

8.1 Events emitted by S3

S3 writes the following events to `event_outbox` (Section 8.4); the `PublishOutboxEvents` job publishes them to the Redis-backed event bus described in S0 §5.1.

Event	Consumed by	Payload (key fields)
<code>wh.events.s3.goods.received</code>	S4 (cache invalidation)	<code>goods_receipt_id</code> , <code>purchase_order_id</code> , <code>supplier_id</code> , <code>total_value</code> , <code>item_count</code>
<code>wh.events.s3.purchase_order.approved</code>	S4 (cache invalidation)	<code>purchase_order_id</code> , <code>supplier_id</code> , <code>total_amount</code> , <code>approval_tier</code> : 1\ 2\ 3
<code>wh.events.s3.order.finalized</code>	S4 (cache invalidation)	<code>order_id</code> , <code>customer_type</code> , <code>customer_ref_id</code> , <code>subtotal</code> , <code>service_charge</code> , <code>vat</code> , <code>total</code>
<code>wh.events.s3.employee_consumption_period.closed</code>	S4 (cache invalidation)	<code>period</code> , <code>employee_count</code> , <code>total_amount</code>
<code>wh.events.s3.guest.checked_in</code>	S4 (cache invalidation)	<code>reservation_id</code> , <code>room_id</code> , <code>folio_id</code> , <code>check_in_at</code>
<code>wh.events.s3.guest.checked_out</code>	S4 (cache invalidation)	<code>reservation_id</code> , <code>room_id</code> , <code>folio_id</code> , <code>check_out_at</code> , <code>total_charges</code>
<code>wh.events.s3.folio.settled</code>	S4 (cache invalidation)	<code>folio_id</code> , <code>room_id</code> , <code>reservation_id</code> , <code>total_charges</code> , <code>total_paid</code> , <code>service_charge</code> , <code>vat</code> , <code>settled_at</code>

All seven events are consumed exclusively by S4 for dashboard cache invalidation (S0 §5.2, §5.4). The revenue journal entry to S4 is a **separate synchronous call** (Section 8.3); the event is only for invalidating S4's cached reads.

8.2 Events consumed by S3

S3 consumes no cross-system events. Every interaction that, in the original M5/M6/M7 design, would have been a cross-service event (ingredient dispatch triggering M5, F&B charge triggering M7's folio write) is now an in-process method call under ID4 and ID5:

Former cross-service interaction	Now (in-process)
M6 emits <code>OrderFinalized</code> → M5 dispatches ingredients	<code>OrderController::finalize()</code> calls <code>FifoFefoService::dispatch()</code> directly in the same transaction (ID4)
M6 emits F&B payment event → M7 posts charge to folio	<code>BillingController::pay()</code> calls <code>FolioService::postChargeForRoom()</code> directly in the same transaction (ID5)

Former cross-service interaction	Now (in-process)
M6 and M7 each post GL entries to M8 independently	S3 sends one netted journal to S4 per folio settlement; daily F&B summary excludes hotel-guest orders (ID3)

S3 still runs the standard `redis-bus-subscriber` worker process required by D2/D8 (so the worker fleet is uniform across all four systems), but it currently has **no event subscriptions registered** — it is idle by design.

8.3 Synchronous calls

Direction	Call	Auth	Idempotent?	Notes
S3 → S4	<code>POST /api/v1/journal-entries</code>	Service-Key (D9)	Yes (D5)	Four journal types: AP/goods-receipt + COGS, folio-settlement (netted, ID3), daily F&B sales summary (excludes hotel_guest, ID3), event-AR journal
S3 → S2	<code>POST /api/v1/employees/{id}/deductions</code>	Service-Key (D9)	Yes (D5)	ID12 — staff meal deduction push at period close; Idempotency-Key = <code>s3-meal-{employee_id}-{period}</code>
S4 → S3	<code>GET /api/v1/rooms, /reservations, /orders, /items, /purchase-orders</code>	Service-Key (D9)	n/a (read-only)	Cached by S4 at 60–120s TTL (S0 §5.4); invalidated by the events in Section 8.1

8.4 Outbox mechanics

S3's `event_outbox` table and `PublishOutboxEvents` job follow the same pattern as S1 and S2 (D2, ID10):

- Within the same database transaction that writes the business change (e.g. `GoodsReceiptController::store()`), S3 inserts a row into `event_outbox` with `status = pending`.
- `PublishOutboxEvents`, scheduled every **10 seconds**, selects `pending` rows, publishes each to the Redis event bus topic matching `event_name`, and marks it `published`.
- If publishing fails, `attempts` is incremented and the row remains `pending` for retry; after a configured maximum (5 attempts with backoff per D4), it is marked `failed` for manual investigation and a `super_admin` notification is fired.

This guarantees that an event is never lost due to a crash between the business write and the publish.

8.5 Idempotency mechanics (D5)

The `idempotency_keys` table (Section 3.4) protects the following endpoints:

Endpoint	Direction	Behaviour on retry with same key
	Inbound	

Endpoint	Direction	Behaviour on retry with same key
PUT /purchase-orders/{id}/approve		Returns cached response; does not re-advance the status or re-post the AP journal
POST /suppliers/{id}/payments	Inbound	Returns cached response; does not create a duplicate <code>supplier_payments</code> row
POST /bills/{id}/payments	Inbound	Returns cached response; does not create a duplicate <code>bill_payments</code> row or re-trigger routing
POST /folios/{id}/payments	Inbound	Returns cached response; does not create a duplicate <code>folio_payments</code> row or re-post the folio-settlement journal
POST /api/v1/employees/{id}/deductions (outbound to S2)	Outbound	S2 returns the cached response from the first successful call (ID12, Section 5.8)

`EnsureIdempotent` middleware on inbound endpoints: (1) looks up `idempotency_keys` by the `Idempotency-Key` header; (2) if found and `request_hash` matches, replays `response_body` / `status_code`; (3) if found with a different `request_hash`, returns `409 IDEMPOTENCY_KEY_CONFLICT`; (4) if not found, proceeds and records the response on completion. `PurgeIdempotencyKeys` runs daily at 02:00 and deletes rows past `expires_at`.

9. Procurement, Supplier Payments, Mandatory Invoice Fields & Cashier Reconciliation

This section covers the S3 workflows that do not fit neatly into the data-model or API tables above — the procurement cycle, supplier payment GL postings, the statutory invoice-field requirements for folios, and the cashier shift reconciliation process that spans both the Restaurant and Hotel modules.

9.1 Purchase order and goods receipt workflow

The full procurement cycle for a stock item:

1. **Draft PO** — `PurchaseOrderController::store()` creates a `purchase_orders` row (`status = draft`) with one or more `purchase_order_lines` rows. `total_amount` is computed as the sum of `quantity_ordered × unit_price` across all lines.
2. **Submit** — `PUT /purchase-orders/{id}/submit` advances `status` to `pending_dept_head` regardless of amount. This is the mandatory first step for every PO.
3. **Tier approval** — `PurchaseOrderController::approve()` advances the status through the appropriate tier(s) (Section 5.3). Each tier is a separate API call by the relevant approver.
4. **Goods receipt** — `POST /purchase-orders/{id}/goods-receipts` records a delivery. `GoodsReceiptController::store()` runs a single transaction (Section 3.1, `goods_receipts` note): creates the receipt, receipt lines, and stock batches; increments `items.current_stock`; updates `purchase_order_lines.quantity_received`; moves the PO to `partially_received` or `closed`; writes `stock_movements` (receipt); and writes `wh.events.s3.goods_received` to `event_outbox`.
5. **Multiple deliveries** — a PO with `status = partially_received` can receive further goods-receipt calls until all lines are fully satisfied and the status becomes `closed`.

CONVENTION

PO immutability after approval. Once `purchase_orders.status` leaves `draft` (i.e. after `submit`), `purchase_order_lines` rows may not be added, deleted, or have `quantity_ordered` changed. If the order changes after submission, the PO must be cancelled and a new one created. This preserves the integrity of the approval audit trail.

9.2 Supplier payment modes and GL postings

`PaymentController` records a `supplier_payments` row and calls `FinanceClientService` to post the AP journal to S4. The journal always reduces `2001 AP — Suppliers` (DR) and credits the appropriate clearing or settlement account (CR):

Payment Method	<code>payment_method</code> value	GL Journal
Cash	<code>cash</code>	Dr 2001 AP — Suppliers / Cr 1001 Cash
Bank transfer	<code>bank_transfer</code>	Dr 2001 AP — Suppliers / Cr 1002 Bank (CBE)
Mobile banking	<code>mobile_banking</code>	Dr 2001 AP — Suppliers / Cr 1003 Mobile Banking Clearing
POS terminal	<code>pos</code>	Dr 2001 AP — Suppliers / Cr 1004 POS Clearing

Payment Method	<code>payment_method</code> value	GL Journal
VISA / international card	<code>visa</code>	Dr 2001 AP — Suppliers / Cr 1005 VISA Clearing
Credit (deferred)	<code>credit</code>	Recorded in <code>supplier_payments</code> ; journal is posted on the agreed settlement date, not at time of recording

After a successful `POST /api/v1/journal-entries` response from S4, `supplier_payments.posted_to_finance` is set to `true`. If the S4 call fails, the row remains `posted_to_finance = false` for retry — the retry is safe because the journal call to S4 carries an `Idempotency-Key` derived from `supplier_payment.id`.

9.3 Mandatory folio invoice fields

`GET /folios/{id}/invoice` generates the structured invoice response (also renderable as a PDF via `InvoicePdfService`). The following 13 fields are mandatory on every generated invoice; the endpoint must return a `422 UNPROCESSABLE` response if any required source column is null at generation time.

#	Invoice Field	Source Column	Notes
1	Room Number	<code>rooms.room_number</code>	From the reservation's room
2	Invoice Number	<code>room_folios.folio_number</code>	Unique, auto-generated at folio creation
3	Guest Full Name	<code>guest_profiles.full_name</code>	
4	Check-In Date	<code>reservations.actual_check_in</code>	Must be set (guest must be checked in)
5	Check-Out Date	<code>reservations.actual_check_out</code>	Must be set (guest must be checked out or checking out now)
6	Total Charges	<code>room_folios.total_charges</code>	Sum of all <code>folio_charges.net_amount</code>
7	Total Paid	<code>room_folios.total_paid</code>	Sum of all <code>folio_payments.amount</code>
8	Outstanding Balance	<code>room_folios.outstanding_balance</code>	<code>total_charges - total_paid</code>
9	Payment Method(s)	<code>folio_payments.payment_method</code> (all rows)	Listed per payment row; multiple methods are shown if partial payments used different methods
10	Cashier Name	Resolved from S1 <code>name</code> JWT claim or S4 cached <code>GET /employees/{cashier_id}</code>	Required on every transaction line
11	Transaction Date/Time	<code>folio_charges.charged_at</code>	Required on every charge line

#	Invoice Field	Source Column	Notes
12	Service Charge Amount	<code>folio_charges.service_charge_amount</code>	Required on every charge line
13	VAT Amount	<code>folio_charges.vat_amount</code>	Required on every charge line

CONVENTION

Mandatory SC and VAT on every folio charge. Every `folio_charges` row — regardless of `service_category` (room rent, F&B, laundry, minibar, event, or other) — must include non-zero `service_charge_amount` and `vat_amount`. `FolioService::addCharge()` enforces this by always applying the SC/VAT formula (Section 5.4) and storing the rate snapshots. An invoice generation request that finds a `folio_charges` row with null or zero SC/VAT where one is expected should raise a reconciliation alert, not silently produce an incorrect invoice.

9.4 Cashier shift reconciliation

`cashier_shifts` (Section 3.3) consolidates a cashier's Restaurant and Hotel cash takings into one shift record. The workflow:

1. **Open shift** — `POST /cashier-shifts` creates a shift (`status = open`). The cashier counts and records `opening_cash_float`.
2. **During the shift** — every `folio_payments` row (Hotel) has `shift_id` set to the open shift (non-nullable). Every `bill_payments` row (Restaurant) has `shift_id` set if the cashier is operating with an open shift (nullable, but expected to be set for all cash transactions).
3. **Close shift** — `POST /cashier-shifts/{id}/close`: the cashier counts and records `closing_cash_counted`; `CashierReportController::closeShift()` computes `expected_cash` as the sum of all `folio_payments.amount` + `bill_payments.amount` where `payment_method = cash` and `shift_id = {id}`, plus `opening_cash_float`; sets `variance = closing_cash_counted - expected_cash`; sets `status = closed`.
4. **Shift report** — `GET /cashier-shifts/{id}/report` returns a breakdown of the shift's transactions grouped by `payment_method` (for both `folio_payments` and `bill_payments`) so the supervisor can verify each payment-method total. The cash total reconciles against the physical cash count via `variance`.

NOTE

Non-cash payment methods (bank transfer, POS, VISA, mobile banking) are included in the shift report for completeness but do not contribute to `expected_cash` or `variance` — those are settled via the respective clearing accounts posted to S4. Only `payment_method = cash` is counted in the physical reconciliation.

10. Security, Secrets & Operational Conventions

10.1 Platform conventions (D1–D14) as applied to S3

ID	Convention	How S3 applies it
D1	One Laravel app, one database per system	<code>wh_s3_db</code> , 29 tables (27 business + <code>event_outbox</code> + <code>idempotency_keys</code>)
D2	<code>event_outbox</code> + <code>PublishOutboxEvents</code> (every 10s)	Section 8.4; 7 emitted event types (Section 8.1)
D3	Event envelope <code>{"event", "version", "occurred_at", "source_system", "request_id", "payload"}</code>	All 7 event types include the standard envelope (S0 §9 D3)
D4	<code>permissions[]</code> claim drives authorization	<code>CheckPermission</code> middleware (Section 6); with additional tier check in <code>PurchaseOrderController::approve()</code>
D5	<code>Idempotency-Key</code> + <code>idempotency_keys</code> table	Section 3.4, Section 8.5 — protects PO approval, supplier payments, bill payments, folio payments, and the outbound S2 deduction push (ID12)
D6	<code>?page= / ?per_page=</code> pagination on list endpoints	All <code>GET</code> list endpoints (Section 4)
D7	Standard error envelope <code>{"error": { "code", "message", "details" } }</code>	All endpoints; <code>422 UNPROCESSABLE</code> is used for folio settlement check-out guard and invoice generation
D8	<code>redis-bus-subscriber</code> worker present on every system	Running but idle on S3 (Section 8.2)
D9	Service-Key authentication for inter-system calls	<code>ServiceKeyAuthenticate</code> — used by outbound calls to S4 and S2, and inbound calls from S4's cached reads
D10	Structured JSON logging	Standard Laravel logging channel; all financial transactions log <code>folio_id / bill_id / po_id</code> + acting <code>cashier_id / requested_by</code> for audit
D11	Health-check endpoint	<code>GET /api/v1/health</code> (S0 §10)
D12	Cross-system contract test pairs	S3 ↔ S1 (JWT auth), S3 ↔ S2 (deduction push, ID12), S3 ↔ S4 (journal entries + 7 event types), S4 ↔ S3 (read-only cached GETs) — Section 11
D13	Soft delete — no hard DELETE	<code>items.is_active</code> , <code>suppliers.is_active</code> , <code>dining_tables.is_active</code> ,

ID	Convention	How S3 applies it
		<code>menu_items.is_available</code> , <code>purchase_orders.status = cancelled</code> (terminal), <code>restaurant_orders.status = cancelled</code> (terminal), <code>reservations.status = cancelled/no_show</code> (terminal)
D14	Backup / DR	Nightly encrypted <code>mysqldump</code> of <code>wh_s3_db</code> ; 30 days local + 90 days offsite; <code>innodb_encrypt_tables=ON</code> ; quarterly DR restore drill logged to S1's <code>audit_logs</code> (S0 §9 D14)

10.2 Environment configuration (`.env`)

```
# Database
DB_DATABASE=wh_s3_db
DB_USERNAME=wh_s3_app
DB_PASSWORD=<secret>

# JWT / Gateway (S0 §4)
JWT_PUBLIC_KEY_PATH=/secrets/jwt/public.pem
SERVICE_KEY=<secret>

# Redis / event bus (D2, D8)
REDIS_HOST=redis.internal
REDIS_PORT=6379
EVENT_OUTBOX_PUBLISH_INTERVAL=10

# Service & Charge configuration (Section 5.4)
SC_RATE=0.10
VAT_RATE=0.15

# PO approval thresholds (Section 5.3)
PO_DEPT_HEAD_THRESHOLD=5000
PO_FINANCE_THRESHOLD=50000

# Inventory alert configuration (Section 7.2)
EXPIRY_ALERT_DAYS=7

# Idempotency (D5)
IDEMPOTENCY_KEY_TTL_HOURS=48

# S4 journal endpoint (Section 8.3)
S4_BASE_URL=http://wh-s4:8000
S4_JOURNAL_PATH=/api/v1/journal-entries

# S2 deduction endpoint (ID12, Section 5.8)
S2_BASE_URL=http://wh-s2:8000
S2_DEDUCTIONS_PATH=/api/v1/employees/{id}/deductions
```

CROSS-REFERENCE

For the full deployment topology — container layout, nginx gateway routing rules, secret management, and shared infrastructure (Redis, MySQL hosts) — see **S0 §10 (Deployment & Operational Conventions)**.

11. Non-Functional Requirements & Testing

Category	Requirement
Availability	S3 must be available 24/7 (the hotel operates around the clock) with 99.5% uptime; the <code>RaiseExpiryAlerts</code> job (07:00) and <code>PublishOutboxEvents</code> / <code>PurgeIdempotencyKeys</code> jobs may run during low-traffic hours but must not block the API
Latency	<code>GET /rooms</code> , <code>/reservations</code> , <code>/orders</code> (list endpoints) must respond in < 300ms at p95; <code>POST /orders/{id}/finalize</code> (ingredient dispatch + bill computation) must complete in < 2 seconds for orders up to 20 items; <code>POST /reservations/{id}/check-in</code> (folio creation + nightly room-rent charges) must complete in < 1 second
Throughput	Must support at least 30 concurrent cashier/receptionist sessions during peak check-in periods (07:00–10:00) and dinner service (19:00–22:00)
Security	All write endpoints involving money (PO approval, supplier payments, bill payments, folio payments, employee deduction push) must be Idempotency-Key protected (D5) and logged with the acting <code>cashier_id</code> / <code>requested_by</code> for audit; inbound reads from S4 must be Service-Key authenticated (D9)
Data integrity	<code>room_folios.outstanding_balance</code> must always equal <code>total_charges - total_paid</code> ; <code>items.current_stock</code> must always equal the sum of all active <code>stock_batches.quantity_remaining</code> for that item; <code>bills.outstanding_balance</code> must always equal <code>total_amount - paid_amount</code>
Testing — unit	<code>FifoFefoService</code> (all FIFO/FEFO dispatch scenarios including <code>InsufficientStockException</code>), <code>BillingService</code> (SC/VAT formula for all 7 customer types), <code>FolioService::addCharge()</code> and <code>recordPayment()</code> (including the <code>outstanding_balance = 0</code> trigger), <code>EmployeeConsumptionService::pushToPayroll()</code> (idempotency and retry logic)
Testing — feature	Full Laravel feature tests for every endpoint in Section 4, covering permission checks (Section 6), lifecycle transitions (Section 7), and the PO approval-tier enforcement
Testing — contract	S3's <code>openapi.json</code> validated against the sync-call contracts in S0 §5.3: <code>POST /journal-entries</code> (outbound to S4, all four journal types), <code>POST /employees/{id}/deductions</code> (outbound to S2, ID12), <code>GET /rooms / reservations / orders / items / purchase-orders</code> (inbound from S4)
Testing — event integration	Per D12, contract test pairs: S3 ↔ S4 (all 7 events in Section 8.1 + journal-entry posts), S3 ↔ S2 (employee deduction push, ID12), S3 ↔ S1 (JWT verification on every endpoint)

Category	Requirement
Testing — in-process integration	<code>OrderController::finalize()</code> integration test must verify that ingredient dispatch (ID4), bill computation, and <code>order.finalized</code> event are all written or applied in a single atomic transaction — a simulated <code>InsufficientStockException</code> mid-dispatch must roll back the bill creation and <code>stock_movements</code> writes
Testing — UAT	Tracked in S4's Requirements Traceability Matrix (RTM) alongside the other three systems

12. Integration Contract Summary

12.1 What S3 provides

To	What	Reference
S4	Seven domain events for dashboard cache invalidation (<code>goods.received</code> , <code>purchase_order.approved</code> , <code>order.finalized</code> , <code>employee_consumption_period.closed</code> , <code>guest.checked_in</code> , <code>guest.checked_out</code> , <code>folio.settled</code>)	Section 8.1, S0 §5.2
S4	Journal entries (synchronous, idempotent): AP/goods-receipt + COGS, folio-settlement (netted, ID3), daily F&B sales summary (ID3), event-AR	Section 8.3, S0 §5.3, S0 §8.1
S4	Read-only cached endpoints: <code>GET /rooms</code> , <code>/reservations</code> , <code>/orders</code> , <code>/items</code> , <code>/purchase-orders</code> (60–120s TTL)	S0 §5.3–5.4
S2	<code>POST /employees/{id}/deductions</code> for staff F&B consumption at period close (ID12)	Section 5.8, Section 8.3

12.2 What S3 depends on

From	What	Reference
S1	JWT verification for every inbound request (via the gateway); <code>permissions[]</code> claims for authorization; <code>name</code> claim for resolving cashier display names (ID2)	S0 §3–4, Section 6
S2	Timely employee deduction acknowledgement at period close — if S2 is down, <code>EmployeeConsumptionService::pushToPayroll()</code> retries using the deterministic <code>Idempotency-Key</code> ; the deduction is never lost, only delayed	Section 5.8, ID12
S4	<code>POST /journal-entries</code> endpoint — if S4 is unavailable, <code>FinanceClientService</code> retries with the idempotent key; <code>supplier_payments.posted_to_finance</code> and folio settlement remain pending until S4 confirms	Section 8.3

12.3 Build-order implication

Because S1 issues every JWT that S3's endpoints require, **a stub or real S1 deployment must exist before S3's endpoints can be exercised end-to-end**. Within S3, the recommended build order is:

1. **Inventory & Purchasing module first** — no internal dependencies; `items`, `suppliers`, and the PO approval chain are prerequisite data for Restaurant and Hotel.
2. **Restaurant & F&B module second** — depends on `items` (via `recipe_ingredients.item_id`) and the `FifoFefoService` (ID4); the `employee_consumption` table and ID12 deduction push can be validated against S2 early.
3. **Hotel & Room Management module third** — depends on `FolioService::postChargeForRoom()` receiving calls from the Restaurant module (ID5); the `cashier_shifts` table integrates both Restaurant `bill_payments` and Hotel `folio_payments`.

The S3 ↔ S4 journal-entry contract (Section 8.3) should be agreed with the S4 developer early — all four journal types are exercised from S3's first goods-receipt and first folio-settlement operations.

End of Document — S3, Hospitality Operations, v1.0